



ODEROM

Oriented Differential Exterior calculus for Relativity, Operators and Manifolds

Manual do usuário · caderno de cálculo tensorial

Rafael Camargo Rodrigues de Lima

Etapa 3 (caderno) concluída · todos os capítulos verificados contra o código-fonte

Sumário

1. Sobre este manual e o ODEROM **completo**
2. Instalação e primeira execução **completo**
3. Anatomia do caderno **completo**
4. Executando blocos: o modelo de teclado **completo**
5. Estados de um bloco **completo**
6. Execução única e cancelamento **completo**
7. Criar, editar e apagar blocos **completo**
8. Salvar e abrir: o formato `%%` **completo**
9. A linguagem ODEROM **completo**
10. Exemplos completos **completo**
11. Uso pela linha de comando **completo**
12. Referência rápida **completo**
13. A galeria de espaços-tempo **completo**

1. Sobre este manual e o ODEROM

O ODEROM é um aplicativo de mesa para cálculo simbólico em geometria diferencial. A partir da definição de uma variedade, uma carta de coordenadas e uma métrica, ele computa objetos de curvatura — símbolos de Christoffel, tensores de Riemann e Ricci, escalares invariantes como o de Kretschmann — em forma fechada, sem avaliação numérica. A interface principal é um *caderno*: blocos de código empilhados que o usuário edita e executa individualmente, com a saída matemática tipografada logo abaixo de cada bloco.

Este documento cobre o caderno e seu modelo de interação, a instalação, o formato de arquivo, a gramática completa da linguagem, quatro exemplos executados de ponta a ponta e o uso pela linha de comando.

Como este manual foi verificado. A gramática, a lista de consultas, as mensagens de erro e os quatro exemplos do capítulo 10 vêm do parser real (`oderom-cli::parser / expr_parser`), de arquivos `.od` já existentes no repositório, ou de arquivos escritos para esta revisão e efetivamente executados pelo binário `oderom` ou pelo `oderom-repl` — nunca de memória. Onde algo não pôde ser verificado desse jeito (por exemplo, pré-requisitos de build em macOS/Windows, que este repositório não exercita em CI), o texto diz isso explicitamente em vez de supor.

2. Instalação e primeira execução

O ODEROM é um workspace Cargo multi-crate; o aplicativo de caderno (`oderom-app`) é um shell Tauri 2 sobre um front-end estático — sem Node.js, sem `npm`, sem passo de build de JavaScript. `tauri.conf.json` não declara `devUrl` nem `beforeBuildCommand`: `build.frontendDist` aponta direto para `oderom-app/dist/`, e CodeMirror 5 e KaTeX estão vendorizados ali como arquivos (não CDN, não dependência npm) — a janela abre sem acesso à rede.

Pré-requisitos

- **Toolchain Rust estável** (edição 2021, `Cargo.toml` do workspace). Nenhuma versão mínima é fixada em `rust-toolchain.toml` — qualquer `stable` recente serve.
- **Linux** — bibliotecas de sistema do WebKitGTK, confirmadas contra o próprio pipeline de CI deste projeto (`.github/workflows/ci.yml`, variável `TAURI_APT_DEPS`), não de documentação genérica do Tauri:

```
sudo apt-get install libwebkit2gtk-4.1-dev build-essential curl wget file \
libxdo-dev libssl-dev libayatana-appindicator3-dev librsvg2-dev
```

- **macOS/Windows** — este repositório não roda CI nessas plataformas (o workflow só tem jobs `ubuntu-latest`), então não há prerequisite verificado por este projeto para citar aqui. Valem os requisitos padrão de qualquer aplicativo Tauri 2 (Xcode Command

Line Tools no macOS; WebView2 Runtime e as ferramentas de build do MSVC no Windows) — não confirmados contra este repositório especificamente.

- **Não é preciso Node.js** em nenhuma plataforma — ver acima.

Fluxo de desenvolvimento (verificado)

O comando real, executado e confirmado nesta revisão:

```
cargo build -p oderom-app
./target/debug/oderom-app
```

Produz um binário executável chamado `oderom-app` em `target/debug/` (ou `target/release/` com `--release`) — sem instalador, sem empacotamento, a janela abre diretamente. É o mesmo fluxo que `cargo tauri dev` executaria (não há `beforeDevCommand` / `devUrl` configurados, então não existe um segundo processo de front-end para orquestrar).

Empacotamento distribuível (Linux) — verificado ao vivo

Desde esta revisão, `cargo tauri build` empacota de verdade: o único obstáculo real era o `identifier` em `oderom-app/src-tauri/tauri.conf.json`, que ainda era o valor-modelo do próprio Tauri (`com.tauri.dev`, recusado por não ser único). Trocado para `com.oderom.notebook` — reverse-DNS, evita de propósito terminar em `.app` (o próprio `tauri-cli` avisa que esse sufixo colide com a extensão de bundle do macOS, um problema que só apareceria numa rodada futura de empacotamento para lá, mas que não custa nada evitar agora) — os três alvos Linux que o `tauri-cli` reconhece (`deb`, `rpm`, `appimage`) empacotam sem mais nenhuma mudança:

```
cargo tauri build -b deb
cargo tauri build -b rpm
cargo tauri build -b appimage
```

Nenhum dos três introduz dependência de Node.js/ `npm` — o passo de bundling é inteiramente nativo (compilação Rust normal, mais ferramentas nativas de empacotamento Linux; a primeira build de `.AppImage` baixa um runtime binário do GitHub uma única vez, não um passo de front-end). Metadados do pacote (nome de exibição, categoria, descrição, licença), até esta revisão ausentes ou herdados de valores internos do `Cargo.toml`, foram preenchidos explicitamente em `tauri.conf.json`:

Campo	Valor
<code>identifier</code>	<code>com.oderom.notebook</code>
<code>productName</code>	<code>ODEROM</code>
<code>bundle.category</code>	<code>Education</code> (o enum do Tauri não tem uma categoria "Science" dedicada)
<code>bundle.shortDescription</code>	Symbolic differential geometry notebook for general relativity

<code>bundle.license</code>	MIT OR Apache-2.0 (mesma licença do workspace Cargo — o bundler não herda esse campo do <code>Cargo.toml</code> automaticamente, precisou ser declarado à parte)
Maintainer/publisher (<code>.deb</code>)	Não fixado — cai no próprio padrão do Tauri (o segundo elemento do <code>identifier</code> , <code>oderom</code>), já que <code>Cargo.toml</code> não declara <code>authors</code> . Decisão consciente desta revisão, revisável depois.

Instalando a partir do pacote (o caminho do usuário final, sem Rust instalado):

```
# .deb (Debian/Ubuntu e derivados)
sudo apt install ./ODEROM_0.1.0_amd64.deb
# ou, sem resolver dependências automaticamente:
sudo dpkg -i ODEROM_0.1.0_amd64.deb

# .AppImage (qualquer distribuição Linux moderna, sem instalar nada)
chmod +x ODEROM_0.1.0_amd64.AppImage
./ODEROM_0.1.0_amd64.AppImage

# .rpm (Fedora/openSUSE e derivados)
sudo rpm -i ODEROM-0.1.0-1.x86_64.rpm
```

O `.deb` / `.rpm` instalam um binário do sistema mais um atalho de menu (`.desktop`), e dependem de `libwebkit2gtk-4.1-0` / `libgtk-3-0` já estarem presentes (declarado como `Depends` no `.deb`). O `.AppImage` não instala nada — é um único arquivo executável autocontido (~77 MB, contra ~4 MB do `.deb` / `.rpm`, porque empacota o runtime GTK/WebKit inteiro junto) — e foi o caminho usado para verificar esta revisão: copiado para um diretório fora da árvore do projeto, executado com `PATH` reduzido a `/usr/bin:/bin` (sem `cargo` / `rustc`) e uma `$HOME` descartável, abriu normalmente — o mesmo caderno com o exemplo de Reissner-Nordström carregado, título "ODEROM" na janela, sem nenhuma saída de erro.

Assinatura de código: nenhum dos três pacotes é assinado, e isso não bloqueia nada no Linux — `.deb` / `.rpm` / `.AppImage` não assinados instalam e abrem normalmente. Fica pendente, conhecido, para quando houver uma rodada de empacotamento macOS/Windows: sem assinatura, o Gatekeeper do macOS e o SmartScreen do Windows avisam ou bloqueiam a abertura — não resolvido aqui, só registrado.

macOS/Windows: não tentados nesta revisão — o próprio pipeline de CI deste projeto só roda `ubuntu-latest` (`.github/workflows/ci.yml`), então não há como validar um empacotamento para essas plataformas a partir daqui; `cargo tauri build` para `.dmg` / `.app` ou `.msi` / `.nsis` precisa rodar de verdade num host macOS/Windows (nativo ou um runner de CI correspondente), não é algo que cross-compile de um Linux resolva. Além da assinatura de código (parágrafo acima), nada mais foi investigado para essas duas plataformas nesta revisão.

Ícones

`oderom-app/src-tauri/icons/` guarda os arquivos rasterizados (`icon-32/128/256/512.png` , `icon.icns` , `icon.ico`), referenciados por `tauri.conf.json`'s `bundle.icon` e gerados a partir de `assets/icon-launcher-1024.svg` (ver `assets/README.md`).

Primeira abertura (verificado ao vivo)

A janela abre com o título "ODEROM", cabeçalho escuro com o símbolo da marca, e o caderno já carregado com o exemplo de Reissner-Nordström embutido em `seed_example()` (`oderom-app/src-tauri/src/lib.rs`) — quatro blocos, todos *não executados* (`[ ]` na medianiz), o primeiro já com foco:

```
manifold M dim 4
bundle TM on M dim 4
%%
chart schw on M coords (t, r, theta, phi)
%%
metric g on schw bundle TM {
  [t,t] = -(1 - 2*M/r + Q^2/r^2),
  [r,r] = 1/(1 - 2*M/r + Q^2/r^2),
  [theta,theta] = r^2,
  [phi,phi] = r^2 * sin(theta)^2
}
%%
kretschmann
```

A barra de status mostra "pronto · 4 blocos · 0 executados". Nada é calculado até uma tecla de execução ser usada — mesmo no primeiro bloco já focado.

3. Anatomia do caderno

O caderno é uma pilha vertical de blocos. Cada bloco tem três partes:

- **A medianiz (gutter), à esquerda**, com um indicador de execução entre colchetes. `[ ]` vazio significa nunca executado; `[n]` com um número significa que aquele foi o n -ésimo bloco executado na sessão. A numeração reflete a ordem real de execução, não a posição do bloco na página.
- **O editor**, onde se digita o código. Há realce de sintaxe para a gramática do ODEROM.
- **A saída**, que aparece abaixo do editor depois da execução, tipografada matematicamente. Resultados em forma de expressão saem como matemática; algumas saídas combinam matemática com uma linha de resumo textual.

Ao final da pilha há sempre um bloco vazio com borda tracejada. Ele é um alvo permanente para começar a digitar e um lembrete do atalho de execução; não é um bloco "real" até receber conteúdo.

No alto da janela ficam o campo de caminho de arquivo, os botões de salvar e abrir, o botão **Galeria** (capítulo 13), que abre um painel para carregar uma métrica conhecida, e, separados por uma divisória, dois controles para voltar o caderno a um estado limpo:

Limpar execução e **Novo caderno** (ver abaixo). No rodapé há uma barra de status que reflete o estado do caderno em tempo real — inclusive se algum bloco está em execução.

Nada recalcula sozinho. Este é o princípio central do caderno. Abrir um arquivo nunca executa nada; todos os blocos vêm não executados. Nenhuma edição, nenhuma navegação e nenhuma ação de interface dispara cálculo. Só as teclas de execução, acionadas deliberadamente pelo usuário, executam um bloco — e apenas aquele bloco. Os dois botões de reset descritos a seguir seguem a mesma regra: nenhum dos dois executa nada.

Limpar execução e Novo caderno: dois jeitos de recomeçar, dois riscos diferentes

Os dois controles ao lado do botão Galeria levam o caderno a um estado limpo, mas não são a mesma coisa — cada um existe para um risco diferente, por isso são botões separados e visualmente distintos (o segundo em vermelho), nunca um único controle com opções.

Botão	O que faz	O que preserva	Pede confirmação?
Limpar execução	Volta todos os blocos a não-executados ([]) e descarta todo o estado de sessão: cache de cálculo, apelidos declarados, contador de execução (a próxima execução volta a ser [1]), marcas de obsolescência, toda saída exibida.	O texto de cada bloco, intacto.	Não — reversível: basta executar de novo para reobter qualquer resultado.
Novo caderno	Apaga todos os blocos e começa vazio (um único bloco em branco, como um caderno recém-criado), limpando também todo o estado de sessão junto.	Nada do conteúdo anterior.	Sim — destrutivo: o texto apagado não pode ser recuperado.

Limpar execução é o análogo do "reiniciar o kernel": o estado depois de clicar é indistinguível do estado de abrir aquele mesmo texto como um arquivo novo — mesmo texto, nenhuma execução, nenhuma memória de sessão. Por ser inteiramente reversível (o texto nunca se perde), não pede confirmação — pedir seria fricção sem propósito, já que não há nada a proteger.

Novo caderno apaga texto que o usuário escreveu, então pede confirmação — um painel simples (não um diálogo modal bloqueante) explica exatamente o que se perde antes de qualquer coisa acontecer. Só um clique explícito no botão vermelho de confirmação executa a ação; fechar o painel de qualquer outra forma (Escape, clicar fora, o botão Cancelar) cancela sem tocar em nada.

Nenhum dos dois toca em arquivo. Se o caderno atual veio de um arquivo aberto ou salvo, nem **Limpar execução** nem **Novo caderno** mexem nesse arquivo em disco — os dois agem

só no que está na tela. Depois de "Novo caderno", o arquivo original continua exatamente onde estava, pronto para ser reaberto (capítulo 8) se o caderno em branco não for o que se queria.

4. Executando blocos: o modelo de teclado

O ODEROM segue o modelo do Jupyter, não o do Mathematica. Há três teclas de execução, e a diferença entre elas está no que acontece com o foco depois:

Tecla	O que faz
 + 	Executa o bloco atual e move o foco para o bloco seguinte. Se o bloco atual for o último, cria um bloco novo vazio abaixo e move o foco para ele.
 + 	Executa o bloco atual e mantém o foco nele. Não cria nada. É o atalho para iterar no mesmo bloco.
 + 	Executa o bloco atual e insere um bloco novo vazio logo abaixo, mesmo que já exista um bloco depois, movendo o foco para o novo.

Mover o foco *não* executa. O bloco de destino fica focado e continua não executado — nunca há execução em cadeia. Cada execução é um gesto separado.

Ao abrir o caderno, o primeiro bloco recebe foco automaticamente, de modo que as teclas funcionam sem precisar clicar primeiro dentro do editor.

5. Estados de um bloco

A medianiz e a aparência do bloco distinguem cinco estados:

Estado	Indicador	Significado
Nunca executado		O bloco ainda não foi rodado. É o estado de todo bloco ao abrir o caderno.
Executado e atual		Foi executado e seu resultado corresponde ao código atual.
Executado e obsoleto	  âmbar + 	Foi executado, mas o código mudou desde então (ou um bloco acima mudou). O resultado antigo continua visível, marcado como desatualizado.
Em execução	—	O cálculo está em andamento. O bloco oferece a opção de cancelar.
Cancelado	—	A execução foi interrompida pelo usuário. Se havia um resultado anterior, ele permanece na tela como obsoleto.

Obsolescência

Quando um bloco já executado tem seu código editado, ele passa a *obsoleto*: o resultado na tela foi calculado a partir de um texto que não existe mais. Além do próprio bloco,

todos os blocos executados *abaixo* dele também ficam obsoletos, porque podem depender do que mudou.

A marca de obsolescência é puramente informativa — ela nunca recalcula nem apaga nada. O resultado antigo continua legível; ele só passa a ser exibido como antigo. A distinção usa cor âmbar (nunca vermelho, que é reservado para erro), mais sinais não cromáticos — uma tarja lateral e um glifo  na medianiz, mais um rótulo textual acima da saída — de modo a ser legível independentemente de percepção de cor.

A marca some quando o bloco é executado de novo (só ele; os blocos abaixo continuam marcados até serem executados individualmente), ou quando a edição é desfeita e o texto volta a ser exatamente o que era no momento da execução.

6. Execução única e cancelamento

O caderno executa **um bloco de cada vez**. Enquanto uma execução está em andamento, as teclas de execução em qualquer outro bloco são recusadas — não iniciam um segundo cálculo. Essa recusa é sempre sinalizada: um realce momentâneo aparece na barra de status e no bloco que está ocupando a execução, indicando quem está bloqueando. A recusa nunca é silenciosa e nunca usa diálogo modal.

A exclusão vale para qualquer combinação — uma consulta não começa enquanto outra roda, e uma definição também não. O bloco recusado não muda de estado nem perde o resultado anterior.

Uma execução em andamento pode ser **cancelada**. O cálculo realmente para. O bloco cancelado recebe o estado de cancelado — não volta a parecer nunca executado, porque houve uma tentativa. Se o bloco já tinha um resultado de execução anterior, esse resultado continua na tela, marcado como obsoleto.

Enquanto um bloco roda, a interface permanece responsiva: os demais blocos aceitam foco e edição normalmente. Bloquear a *execução* não bloqueia a *edição*.

Como sair de um bloqueio. Se você tentar rodar um bloco e a execução for recusada, há duas saídas: esperar o bloco em andamento terminar, ou cancelá-lo. Assim que a execução em curso termina ou é cancelada, o próximo bloco pode ser executado normalmente.

7. Criar, editar e apagar blocos

Cada bloco traz, no canto superior direito, um único controle: apagá-lo — o mesmo espaço vira o botão de cancelar enquanto o bloco está em execução (capítulo 6). Não existe um botão dedicado para inserir um bloco novo em qualquer ponto: isso só acontece como efeito das teclas de execução — **Shift + Enter** no último bloco, ou **Alt + Enter** em qualquer bloco (capítulo 4). Editar é direto: o texto é um editor de código comum, com realce. Blocos não podem ser reordenados — a ordem de criação é a única forma de decidir a posição na v1.

Inserir ou apagar um bloco marca como obsoletos os blocos executados abaixo do ponto alterado, pela mesma lógica do capítulo 5 — a estrutura mudou, então o que vinha depois pode não corresponder mais.

8. Salvar e abrir: o formato `%%`

Cadernos são salvos e abertos em um formato de texto que separa os blocos por um marcador `%%`. O campo de caminho no alto da janela, com os botões salvar e abrir, controla isso. Um caderno reaberto volta com todos os blocos *não executados* — coerente com "nada recalcula sozinho": abrir um arquivo nunca dispara cálculo.

A forma exata (`oderom-notebook/src/persist.rs`): `%%` delimita blocos como **uma linha cujo conteúdo, depois de remover um `\r` final, é exatamente `%%`** — não um prefixo, não aceita rótulo nem metadado ao lado. N blocos precisam de exatamente N–1 linhas delimitadoras; o primeiro bloco não tem delimitador antes dele. Seguro porque comentário de `.od` é `#`, nunca `%%` — mas se o próprio texto de um bloco contiver uma linha que já seja exatamente `%%`, salvar é **recusado** com erro (não corrompe o arquivo escrevendo mesmo assim).

O tipo do bloco nunca é gravado. Só o texto-fonte de cada bloco é salvo — nunca a saída, nunca se é declaração ou consulta. Ao reabrir, cada bloco começa como se tivesse acabado de ser criado (nunca executado), e seu tipo é redescoberto do zero por `classify_block` a partir do próprio texto — nunca pode ficar dessincronizado do conteúdo real, porque nunca é guardado separadamente dele.

Exemplo real, gerado pela própria função de serialização (`persist::render`) a partir de um caderno de quatro blocos — não reconstruído à mão:

```
manifold M dim 4
bundle TM on M dim 4
%%
chart schw on M coords (t, r, theta, phi)
%%
metric g on schw bundle TM {
  [t,t] = -(1 - 2*M/r),
  [r,r] = 1/(1 - 2*M/r),
  [theta,theta] = r^2,
  [phi,phi] = r^2 * sin(theta)^2
}
%%
kretschmann
```

Extensão de arquivo: o campo de caminho do aplicativo mostra o placeholder `caminho/arquivo.odn` (`oderom-app/dist/index.html`) — `.odn` é a convenção sugerida para um caderno salvo. Isso é só uma dica de interface, não uma validação: `save_notebook` / `open_notebook` (o comando Tauri) aceitam qualquer caminho, sem checar a extensão. Um arquivo `.od` "puro", do tipo que a linha de comando consome (capítulo 11) — sem nenhuma linha `%%` — não é o mesmo uso: ele não tem conceito de bloco nenhum, é uma sequência contínua de declarações lida de uma vez por `parse_model`. Se aberto no

caderno, um arquivo assim vira **um único bloco** com todo o conteúdo junto (`parse_sources` devolve um vetor de um elemento quando não encontra nenhum delimitador) — não um bloco por declaração.

9. A linguagem ODEROM

Um bloco de caderno (ou um arquivo `.od`) contém uma sequência de **declarações** e **consultas**. O próprio parser decide qual é qual olhando só a primeira palavra do bloco (`classify_block`, `oderom-cli/src/parser.rs`) — seis palavras-chave de declaração e doze de consulta, dois conjuntos disjuntos, sem ambiguidade:

```
manifold bundle head chart metric connection      (declaração)
christoffel riemann ricci scalar kretschmann geodesic accel      (consulta)
einstein riccisquare gaussbonnet weyl weylsquare      (consulta)
```

Qualquer outra palavra inicial é um erro explícito, nunca uma adivinhação silenciosa.

9.1 Variedade, fibrado e carta

```
manifold M dim 4
bundle TM on M dim 4
chart schw on M coords (t, r, theta, phi)
```

`manifold NOME dim N` declara uma variedade de dimensão `N`. `bundle NOME on VARIEDADE dim N` declara um fibrado sobre uma variedade já declarada — `dim` aqui é a dimensão da fibra, não precisa coincidir com a da variedade em geral, embora para o fibrado tangente (o único usado nos exemplos deste manual) sempre coincida. `chart NOME on VARIEDADE coords (c1, c2, ...)` declara uma carta de coordenadas; o número de nomes de coordenada tem que bater exatamente com a dimensão da variedade — `chart c on M coords (x, y)` contra uma `manifold M dim 3` é rejeitado com `chart `c` has 2 coordinates, but manifold `M` has dimension 3` (mensagem real, capturada rodando o parser).

9.2 Métrica e conexão

```
metric g on schw bundle TM {
  [t,t] = -(1 - 2*M/r + Q^2/r^2),
  [r,r] = 1/(1 - 2*M/r + Q^2/r^2),
  [theta,theta] = r^2,
  [phi,phi] = r^2 * sin(theta)^2
}
```

`metric NOME on CARTA bundle FIBRADO { LINHA, LINHA, ... }`. `bundle` aqui nomeia qual fibrado já declarado carrega os índices do tensor métrico — a métrica declara, por conta própria, uma cabeça abstrata (Marco 1) de posto 2, simétrica, com os dois índices covariantes; isso acontece automaticamente, nunca é escrito à mão. Cada `LINHA` dentro de `{ }` é `[i,j] = EXPR`, separadas por vírgula, sem vírgula depois da última. Os índices

entre colchetes são nomes de coordenada da carta referenciada (ou um inteiro literal). A forma alternativa em LaTeX usa subscrito em vez de colchetes, ecoando o próprio nome da declaração: `g_{tt} = ...` (ver 9.4) — as duas formas produzem a árvore idêntica; verificado comparando a saída de `kretschmann` rodada contra a mesma métrica escrita das duas formas (`schwarzschild_ascii.od` e `schwarzschild_latex.od`, `oderom-cli/tests/fixtures/`).

```
connection Gamma on flat2 {
  [x,y,y] = x
}
```

`connection NOME on CARTA { LINHA, ... }` — mesma estrutura de bloco, mas declara diretamente os símbolos de Christoffel (posto 3), sem passar por uma métrica: Christoffel não é um tensor, então não há cabeça, não há simetria declarada, é armazenado como uma grade crua. Uma conexão dispensa `bundle` por completo. É a porta de entrada alternativa para `christoffel / riemann / ricci` sem nunca declarar uma métrica — mas `scalar / kretschmann`, que precisam inverter uma métrica para subir índice, recusam com erro nesse caso (ver 9.5 e capítulo 10, exemplo 4).

9.3 `head` (construção abstrata, Marco 1)

```
head R : TM*, TM*, TM*, TM* symmetry (1 2)- (3 4)- (1 3)(2 4)+
head eps : TM*, TM*, TM* symmetry antisymmetric
```

`head NOME : SLOT, SLOT, ... [symmetry GERADOR ...]` declara uma cabeça tensorial abstrata contra um fibrado, sem componentes concretas — o nível de `oderom canon` (capítulo 11), não das cinco consultas de curvatura. Cada `SLOT` é o nome de um fibrado, sozinho (contravariante) ou seguido de `*` (covariante). `symmetry` aceita `antisymmetric`, `symmetric`, ou uma lista de geradores explícitos — cada um uma sequência de ciclos entre parênteses (índices de posição, 1-based) seguida de `+` ou `-` para o sinal. `metric / connection` nunca precisam de `head` escrito à mão — já declaram a sua própria.

9.4 Expressões escalares

A gramática de expressão (`SCALAR_EXPR`, `oderom-cli/src/expr_parser.rs`) é uma só, com duas grafias de token em todo ponto que tem equivalente em LaTeX — nunca dois parsers:

Forma	ASCII	LaTeX
soma, subtração	<code>a + b</code> , <code>a - b</code>	iguais
produto	<code>a * b</code> ou justaposição (<code>2M</code>)	justaposição
divisão	<code>a / b</code>	<code>\frac{a}{b}</code>
potência (expoente inteiro, só)	<code>a^n</code> , <code>a^-n</code> , <code>a^{-n}</code>	<code>a^{n}</code>

agrupamento	<code>(...)</code>	<code>(...)</code> ou <code>\left(... \right)</code>
seno, cosseno	<code>sin(x)</code> , <code>cos(x)</code>	<code>\sin(x)</code> , <code>\sin^2(x)</code> , <code>\sin{x}</code> , <code>\cos(...)</code>
exponencial	<code>exp(x)</code>	<code>\exp(x)</code>
seno e cosseno hiperbólicos	<code>sinh(x)</code> , <code>cosh(x)</code>	<code>\sinh(x)</code> , <code>\cosh^2(x)</code> , ...
letra grega	identificador literal (<code>theta</code>)	<code>\theta</code> , <code>\phi</code> , ...

`sin / cos / exp / sinh / cosh` são as únicas funções *conhecidas* — com valor e derivada já definidos pelo motor. A forma `NOME(...)` é sempre sintaticamente reservada para chamada de função (nunca lida como justaposição); um nome fora dessa lista de cinco é uma de duas coisas, nunca um erro genérico: se for um nome sem significado matemático padrão (`f` , `g` , `h` , ...), é uma **função indeterminada** (seção 9.10) — um símbolo opaco, aceito e representável, mas sem valor nem derivada conhecidos a priori; se for uma função elementar conhecida ainda não implementada como chamável (`tan` , `sqrt` , uma trigonométrica inversa, ...), é rejeitado explicitamente — nunca aceito como se fosse indeterminado (seção 9.10 tem a lista completa e o porquê). O expoente de `^` é sempre um inteiro literal (positivo ou negativo) — nunca uma subexpressão.

`exp` é sua própria derivada; `sinh / cosh` derivam um no outro, sem troca de sinal (ao contrário de `sin / cos`). `sinh / cosh` satisfazem a identidade cruzada $\cosh(x)^2 - \sinh(x)^2 = 1$ — a mesma simplificação que `sin / cos` já fazem com $\sin(x)^2 + \cos(x)^2 = 1$, sinal trocado — verificado ao vivo: a métrica do plano hiperbólico do capítulo 10 só reduz a um escalar de Ricci constante (`-2`) por causa dessa identidade. `exp` não ganhou a identidade $\exp(a) \cdot \exp(b) = \exp(a+b)$: nenhum caso de uso real deste projeto precisa dela (potências inteiras do mesmo `exp(...)` já se combinam sozinhas, sem identidade nenhuma), e ela exigiria mesclar dois átomos diferentes em vez de só reduzir a potência de um — uma peça de maquinaria nova, não uma extensão pequena do que já existe.

9.5 Consultas

`QUERY := NOME_DA_CONSULTA IDENTIFICADOR? MARCADOR_DE_VARIANCIA?` — a palavra-chave sozinha, ou seguida do nome de uma métrica/conexão declarada, para desambiguar quando há mais de uma (`kretschmann g`), e opcionalmente um marcador de variância de índice entre colchetes por último (`riemann g [up,down,down,down]` — seção 9.16, sua própria sintaxe/regras). Sem o identificador, a mesma regra de `resolve_choice` vale: se há exatamente uma métrica (ou conexão, na ausência de métrica), ela é usada; mais de uma sem desambiguar é erro. `geodesic / accel` (seções 9.11/9.12) são as duas exceções a essa gramática — cada uma exige um segundo identificador, sempre por último, o nome do parâmetro afim, e nenhuma das duas aceita marcador de variância (produzem equações, não objetos tensoriais indexados). As doze, com o que cada uma retorna — verificado rodando cada uma contra `schwarzschild_ascii.od` , `reissner_nordstrom.od` , `hyperbolic_plane.od` e `connection_only.od` (capítulo 10, exemplos 4–8):

Consulta	Retorna	Funciona a partir de <code>connection</code> sem métrica?
<code>christoffel</code>	Os símbolos de Christoffel Γ^a_{bc} (rótulo <code>Gamma</code>).	Sim.
<code>riemann</code>	O tensor de Riemann. A partir de uma métrica, totalmente covariante, R_{abcd} (o primeiro índice é baixado); a partir de uma conexão pura, na forma mista R^a_{bcd} , já que não há métrica para baixar índice. Rótulo <code>R</code> nos dois casos.	Sim (na forma mista).
<code>ricci</code>	O tensor de Ricci $R_{bd} = R^a_{bad}$ (rótulo <code>Ricci</code>) — a contração não precisa inverter a métrica.	Sim.
<code>scalar</code>	O escalar de Ricci $R = g^{bd}R_{bd}$ — um número/expressão só, sem rótulo.	Não — erro <code>`scalar` needs a metric to invert (only a connection was declared)</code> .
<code>kretschmann</code>	O escalar de Kretschmann $R_{abcd}R^{abcd}$ — um número/expressão só, sem rótulo.	Não — mesmo erro do <code>scalar</code> .
<code>geodesic</code> (9.11)	As equações da geodésica, uma por coordenada: $\ddot{x}^a + \Gamma^a_{bc}\dot{x}^b\dot{x}^c = 0$. Sem rótulo — uma lista de equações, não de componentes de tensor.	Sim — só precisa de Γ , nunca de g^{ab} , ao contrário de <code>scalar</code> / <code>kretschmann</code> .
<code>accel</code> (9.12)	As mesmas equações de <code>geodesic</code> , resolvidas para a segunda derivada de cada coordenada: $\ddot{x}^a = f^a(x, \dot{x})$. A forma que um integrador numérico consome diretamente. Sem rótulo.	Sim — mesma razão de <code>geodesic</code> : só precisa de Γ .
<code>einstein</code> (9.13)	O tensor de Einstein $G_{ab} = R_{ab} - \frac{1}{2}g_{ab}R$, totalmente covariante, listado por componentes independentes exatamente como <code>ricci</code> (rótulo <code>G</code>).	Não — <code>g_ab</code> aparece de forma literal na própria fórmula, e <code>R</code> já precisa de <code>g^bd</code> ; mesmo erro do <code>scalar</code> / <code>kretschmann</code> , com o nome do comando trocado.
<code>riccisquare</code> (9.14)	O invariante $R_{ab}R^{ab}$ — um número/expressão só, sem rótulo.	Não — precisa subir os dois índices do Ricci.
<code>gaussbonnet</code> (9.14)	A densidade de Gauss-Bonnet/Euler $R_{abcd}R^{abcd} - 4R_{ab}R^{ab} + R^2$ — recombinação pura de <code>kretschmann</code> / <code>riccisquare</code> / <code>scalar</code> , sem rótulo.	Não — herdado dos três escalares que combina.

<code>weyl</code> (9.15)	O tensor de Weyl (conforme) C_{abcd} , totalmente covariante, listado por componentes independentes exatamente como <code>riemann</code> (rótulo <code>C</code>).	Não — <code>g_ab</code> /R aparecem de forma literal na fórmula. Recusa também em exatamente 2 dimensões (seção 9.15).
<code>weylsquare</code> (9.15)	O invariante $C_{abcd}C^{abcd}$ — um número/ expressão só, sem rótulo.	Não — mesma razão de <code>weyl</code> , cuja barreira de dimensão também herda.

Não existe consulta de holonomia na linguagem: a integração numérica de geodésica/ transporte paralelo (Marco 5, `oderom-components::holonomy`) não está conectada a nenhuma palavra-chave de consulta, nem ao caderno, nem à CLI — é exercitada só por um teste Rust dedicado (`oderom-components/tests/holonomy.rs`), com uma malha fixada à mão, não por um caderno. `geodesic` / `accel` (abaixo) são o caminho *simbólico* — escrevem e isolam algebricamente, nunca integram numericamente; os dois nunca se confundem, cada um mora no seu próprio módulo, e um round futuro que muda isso precisa revisar este parágrafo.

Qualquer uma das doze consultas acima pode ser envolvida por `export ALVO` (seção 9.17) para sair na sintaxe de uma ferramenta simbólica externa (Mathematica, SymPy) em vez da tipografia deste projeto — `export sympy kretschmann`, `export mathematica riemann g [up,down,down,down]`. Não é uma décima terceira consulta, é um envoltório aditivo sobre qualquer uma das doze, incluindo o marcador de variância da seção 9.16.

9.6 Alvos de saída

`Target::Unicode` (padrão), `Target::Latex`, `Target::Json`. No caderno/aplicativo, a escolha não é do usuário: cada resultado calcula e guarda os dois textos, unicode e LaTeX, ao mesmo tempo (`EntryResult`, `oderom-session/src/entry.rs`) — a janela sempre tipografa o LaTeX via KaTeX. Só a linha de comando expõe a escolha, pela flag `--target` (capítulo 11). Apesar do nome, `Unicode` não converte nomes gregos para glifo — `theta` continua "theta" (texto simples, "console"); é o alvo `Latex` que produz `\theta`, `\sin\left(...\right)`, `\frac{...}{...}` — verificado rodando a mesma consulta com as duas flags e comparando a saída real:

```
--target unicode:
Gamma[^theta,phi,phi] = -sin(theta)*cos(theta)
--target latex:
\Gamma^{\theta}_{\phi \phi} = -\sin\left(\theta\right) \cos\left(\theta\right)
```

`Target::Json` produz uma árvore serializada da expressão (para um escalar isolado) ou um objeto com o resumo de componentes (para `christoffel` / `riemann` / `ricci`) — não um formato de texto para leitura, um formato para consumo por outro programa; ao contrário dos outros dois alvos, continua com índices inteiros crus (`"indices":[2,3,3]`) — é um formato para outro programa consumir, não para uma pessoa colar num artigo, então não recebeu o tratamento tipográfico abaixo.

`--target mathematica` / `--target sympy` (Rodada Exportação, seção 9.17) também existem como valor de `--target` nas dez consultas de componentes/escalar comuns da CLI — um

jeito rápido de olhar a sintaxe traduzida sem passar pela gramática de `export`. A diferença real: sem a detecção/renomeação de colisão com palavra reservada nem a linha `symbols(...)` de SymPy que `export` acrescenta — `--target sympy` é uma olhada rápida, nunca a garantia de "colou e rodou" que só `export` promete. Para qualquer coisa que precise realmente rodar na ferramenta de destino, use `export`.

9.7 Tipografia de componentes e cópia por clique

Uma consulta que lista componentes (`christoffel / riemann / ricci`) nunca mostra um índice numérico cru — `R_{0,1,0,1}` não aparece em lugar nenhum. Os índices são os próprios nomes de coordenada da carta (letras gregas como `\theta / \phi` no alvo LaTeX, por extenso no Unicode), separados por um espaço simples no LaTeX — `R_{t r t r}` — nunca por vírgula. O rótulo em si passa pela mesma conversão: `christoffel` é `\Gamma`, a letra grega de verdade, não as cinco letras romanas soltas `Gamma` (que o KaTeX tipografaria como se fossem cinco variáveis multiplicadas). O alvo `Unicode`, sem tipografia matemática nenhuma, mantém a vírgula (`R[t,r,theta,phi]`) e o rótulo por extenso (`Gamma`), exatamente como já era.

O espaço entre os nomes no LaTeX não é estético — é o que impede um macro grego de "engolir" a letra seguinte. `\theta` é uma *control word*: o TeX lê a barra invertida e continua consumindo letras enquanto houver letras, então `\theta` direto seguido da letra `r`, sem nada entre os dois, é lido como um único macro `\thetar` — que não existe, e o KaTeX mostra como texto cru em vermelho em vez de tipografar. Um espaço resolve sem alterar o resultado visual (modo matemático ignora espaço em branco para espaçamento — `t r` não abre nenhum vão a mais que `tr` abriria), mas encerra a leitura do macro no lugar certo. Foi exatamente esse bug — `\thetar` aparecendo cru na tela — que expôs os dois problemas junto: o rótulo `Gamma` sem a barra invertida e a concatenação sem separador. Os dois já saíam errados desde a primeira versão deste recurso; só ficaram visíveis quando alguém olhou de perto o resultado de `christoffel` — o exemplo do parágrafo anterior (`\Gamma^{\theta}_{\phi \phi}`) já é a forma corrigida.

O índice é mostrado exatamente na variância em que a componente foi de fato calculada — nunca ajustado por estética. `riemann` a partir de uma métrica já teve o primeiro índice baixado, então sai totalmente covariante, `R_{t r t r}`, todo em subscrito; a partir de uma conexão pura (sem métrica para baixar índice) sai mista, `R^a_{bcd}`, com o índice contravariante em sobrescrito à esquerda — nunca o contrário, e nunca uma forma "arrumada" para combinar com a outra. `christoffel` é sempre mista, `\Gamma^a_{bc}` (um índice em cima, dois embaixo), `ricci` é sempre totalmente covariante, `R_{bd}`, não importa se veio de métrica ou de conexão.

A anotação de quantos índices brutos uma componente independente resume (`(N components by symmetry)`) nunca fica colada na mesma linha da fórmula — é uma frase em inglês comum, nunca matemática, então nunca passa pelo tipografador de LaTeX/KaTeX junto com a fórmula: no caderno/aplicativo aparece como texto pequeno e cinza ao lado da fórmula (nunca dentro do que é copiado); na linha de comando e no REPL, como uma linha própria, indentada, logo abaixo. A mesma separação vale para as linhas de resumo do final (contagem de componentes truncadas, contagem de componentes identicamente nulas) — nunca compartilham linha com uma fórmula real.

No caderno/aplicativo, clicar em qualquer componente de um resultado copia para a área de transferência o LaTeX limpo daquela componente — só a igualdade tipografada ($R_{tr} = \frac{\dots}{\dots}$), pronta para colar direto num documento LaTeX: nunca a anotação de simetria, nunca a linha de resumo, nunca outra componente, nunca o bloco inteiro. Um pequeno rótulo "copiado" aparece por um instante junto da componente clicada como confirmação — não um diálogo modal. Cada resultado, por trás dessa interação, já chega ao JavaScript com a fórmula e a anotação como campos separados (nunca uma única string concatenada) — a decisão de qual texto é fórmula e qual é legenda é feita inteiramente em Rust (`oderom_components::RenderedComponent`), antes mesmo da tela existir; o JavaScript só desenha essa divisão já pronta, nunca a decide.

9.8 Mensagens de erro comuns

Todas capturadas rodando o parser/CLI de verdade contra um arquivo construído para provocar cada uma:

Situação	Mensagem real
Marca de derivada (') numa função conhecida (<code>sin / cos / exp / sinh / cosh</code>)	<code>`sin` is a known function with a known derivative -- write `diff(sin(...), var)`, not `sin'(...)` or `sin_{...}(...)`</code>
Notação de linha (') numa função indeterminada de mais de um argumento	<code>`h` (prime notation) is ambiguous for a function of 2 arguments -- use `h_{var,...}(...)` instead</code>
Subscrito de derivada nomeando uma variável que não é argumento da função	<code>`h`'s derivative subscript names `x`, which is not one of its own arguments</code>
Marca de derivada sem lista de argumentos em seguida	<code>`f` with a derivative mark must be followed by an argument list, e.g. `f'(r)` or `f_{r}(r)`</code>
Carta com número errado de coordenadas	<code>chart `c` has 2 coordinates, but manifold `M` has dimension 3</code>
Nome já declarado	<code>chart `c` is already declared</code>
Nenhuma métrica nem conexão no arquivo	<code>no metric or connection found in the file</code>
Mais de uma métrica, sem desambiguar	<code>the file declares more than one metric (g1, g2); pick one with --metric</code>
<code>scalar / kretschmann / einstein / riccisquare / gaussbonnet / weyl / weylsquare</code> sem métrica disponível	<code>`scalar` needs a metric to invert (only a connection was declared) (idem, nome do comando trocado)</code>
<code>weyl / weylsquare</code> numa métrica de dimensão 2	<code>the Weyl tensor is not defined in 2 dimensions (the standard formula's 1/(n-2) term is undefined at n=2)</code>

Palavra-chave inicial não reconhecida (bloco do caderno)	<code>expected a declaration (manifold/bundle/head/chart/metric/connection), an alias (NOME := expression), or a query (christoffel/riemann/ricci/scalar/kretschmann/geodesic/accel/einstein/riccisquare/gaussbonnet/weyl/weylsquare), found ...</code>
Uso de um nome nunca declarado como alias (nem antes, nem depois, no documento)	nenhuma — permanece uma variável livre comum, como sempre foi
Uso de um alias antes da sua própria declaração	<code>alias `f` is used before its own `f := ...` declaration</code>
Redeclaração do mesmo nome de alias	<code>alias `f` is already declared</code>
<code>geodesic / accel</code> sem nome de parâmetro	<code>`geodesic` needs an affine parameter name, e.g. `geodesic tau` or `geodesic g tau` (idêntica para <code>accel</code>, com o nome do comando trocado na própria mensagem)</code>
<code>geodesic / accel</code> com parâmetro igual a uma coordenada da carta	<code>affine parameter `r` collides with chart coordinate `r` -- geodesic needs a distinct name (ou -- accel needs a distinct name</code> , conforme o comando)
<code>geodesic / accel</code> com parâmetro igual a uma variável livre da métrica/conexão	<code>affine parameter `M` collides with a free variable already used in this metric/connection -- geodesic needs a distinct name</code> (idem, nome do comando trocado para <code>accel</code>)
<code>accel</code> : coeficiente da segunda derivada normaliza para zero	<code>the geodesic equation for coordinate `r` has a second-derivative coefficient that is identically zero -- cannot solve for its acceleration by dividing by zero</code> (nunca disparada por uma métrica/conexão real hoje — seção 9.12)

9.9 Aliases de expressão

```
f := 1 - 2*M/r + Q^2/r^2

manifold M dim 4
bundle TM on M dim 4
chart schw on M coords (t, r, theta, phi)
metric g on schw bundle TM {
  [t,t] = -f,
  [r,r] = 1/f,
  [theta,theta] = r^2,
  [phi,phi] = r^2 * sin(theta)^2
}
```

`NOME := EXPRESSÃO` declara um alias: um nome curto para uma subexpressão escalar que, de outro modo, teria que ser repetida por extenso em toda componente que a usa — o

exemplo acima é a métrica de Reissner-Nordström, com a função $f(r) = 1 - 2M/r + Q^2/r^2$ escrita uma única vez. Um bloco de alias classifica como declaração (mesmo grupo de `manifold / bundle / chart / metric / connection`) e por isso participa do mesmo mecanismo de reconstrução de declarações do caderno: o escopo de um alias é **o caderno inteiro, de cima para baixo, a partir do ponto da sua declaração** — não é local a um bloco. Um alias pode referenciar outro alias declarado antes dele.

O alias é **puro açúcar sintático**, expandido por substituição textual dentro do parser de expressões, antes de qualquer coisa chegar ao motor de normalização (`oderom-expr`). Depois de expandido, não sobra rastro: a árvore de expressão resultante de `[t,t] = -f` é byte a byte idêntica à que `[t,t] = -(1 - 2*M/r + Q^2/r^2)` produziria escrita à mão — verificado comparando o `kretschmann` das duas formas da métrica de Reissner-Nordström (`reissner_nordstrom.od` contra `reissner_nordstrom_alias.od`, `oderom-cli/tests/fixtures/`): mesmo resultado, byte idêntico, `(-96*M*Q^2*r + 48*M^2*r^2 + 56*Q^4)/r^8`. Declarar ou expandir um alias nunca dispara cálculo nenhum por conta própria — só é resolvido quando o bloco que o usa é de fato executado, a mesma regra de sempre ("nada recalcula sozinho").

Um alias é usado **sem parênteses** — só `f`, nunca `f(...)`. A forma com parênteses é sempre chamada de função (seção 9.4/9.10): `f(r)` é a função indeterminada `f` aplicada a `r`, nunca lida como uma chamada de alias, mesmo que `f` esteja *também* declarado como alias no mesmo arquivo — as duas coisas não colidem porque vivem em posições sintáticas diferentes (com parênteses ou sem), nunca a mesma. Um nome de alias tem que ser declarado antes de usado (referência antecipada é erro, não é lida como variável livre); um nome nunca declarado como alias em lugar nenhum do documento continua sendo uma variável livre comum, exatamente como `M`, `Q` ou qualquer outro símbolo físico sempre foi. Redefinir o mesmo nome de alias é erro — a mesma regra que já vale para `chart / metric / connection` com nomes duplicados.

Como um bloco de alias é uma declaração como qualquer outra, a mesma regra conservadora de obsolescência (capítulo 5) já cobre edição de alias sem nenhuma máquina nova: editar um bloco de alias marca obsoleto tudo abaixo dele no caderno, inclusive blocos de métrica/conexão/consulta que o usam — verificado com um teste dedicado (`editing_an_alias_block_marks_a_later_block_that_uses_it_obsolete`, `oderom-notebook/src/notebook.rs`) e também pela suíte de janela real (`oderom-app/dist/keytest.js`).

9.10 Funções indeterminadas e derivadas

Esta seção só **representa** e **deriva** — nunca resolve nada. Não existe consulta nem operação que resolva uma função indeterminada para uma forma fechada, integre uma equação, ou monte a geodésica (isso é trabalho de uma rodada futura, separada; nada aqui antecipa como). O que existe é: escrever `f(r)` sem dar uma definição, e derivar corretamente uma expressão que a contenha.

```
manifold M dim 4
bundle TM on M dim 4
chart schw on M coords (t, r, theta, phi)
```

```
metric g on schw bundle TM {
  [t,t] = -f(r),
  [r,r] = 1/f(r),
  [theta,theta] = r^2,
  [phi,phi] = r^2 * sin(theta)^2
}
```

`f(r)` declara uma **função indeterminada**: um símbolo funcional novo, sem definição, que o motor trata como uma folha opaca — nunca tenta avaliar, nunca expande, nunca inventa uma identidade algébrica ligando `f` a nada. É deliberadamente **diferente** de um alias e de uma função conhecida, e as três nunca colapsam numa só: um alias (`NOME := EXPR`, seção 9.9) expande para uma expressão já conhecida; uma função conhecida (`sin`, `exp`, ...) tem valor e derivada já definidos pelo motor; uma função indeterminada não tem nem uma coisa nem outra — sua única propriedade é *ter* derivadas, que são elas próprias símbolos opacos (`f'`, `f''`), nunca simplificadas além da regra da cadeia. O mesmo nome pode ser as três coisas em posições diferentes sem colisão nenhuma: `f` sozinho é o alias (se declarado), `f(r)` é sempre a função indeterminada, `sin(x)` é sempre a função conhecida — a posição sintática (com ou sem parênteses, e qual nome) decide, nunca uma reinterpretação silenciosa.

Nem todo nome fora da lista de cinco funções conhecidas vira função indeterminada, porém — um **conjunto reservado** de nomes com significado matemático padrão fica de fora dos dois grupos, rejeitado explicitamente em vez de silenciosamente aceito como símbolo opaco. O motivo é concreto: antes dessa reserva existir, `tan(theta)` numa métrica era aceito como função indeterminada — e derivado como `tan'`, um símbolo opaco qualquer, nunca a derivada real da tangente, `sec2`. Para quem escreveu `tan(theta)` esperando a tangente trigonométrica de verdade, o resultado parecia plausível e estava silenciosamente errado — exatamente a classe de falha (um número que passa por certo) que este projeto trata como mais grave que um erro alto. O conjunto reservado hoje: as recíprocas trigonométricas (`tan`, `cot`, `sec`, `csc`) e suas análogas hiperbólicas (`tanh`, `coth`, `sech`, `csch`); as duas grafias comuns de cada função trigonométrica e hiperbólica inversa (`asin` / `arcsin`, `acos` / `arccos`, `atan` / `arctan`, `asinh` / `arcsinh`, `acosh` / `arccosh`, `atanh` / `arctanh`); `log` / `ln` (logaritmo); `sqrt` (raiz quadrada); `atan2` (arco-tangente de dois argumentos, comum em física para recuperar um ângulo); `abs` (valor absoluto). Ficam de fora funções não-diferenciáveis/discretas sem derivada natural (`floor`, `ceil`, `sign`) — fora do escopo de um projeto centrado em curvatura diferenciável, e não a falha que esta lista existe para fechar.

A reserva é **"ainda não", nunca "proibido para sempre"**: cada nome nela é uma função real que este projeto pode vir a implementar como conhecida numa rodada futura (a mesma forma como `exp` / `sinh` / `cosh` já passaram de indisponíveis a conhecidas numa rodada anterior) — reservar o nome agora é o que mantém essa porta aberta sem deixá-lo significar outra coisa (uma função indeterminada opaca) enquanto isso não acontece. Chamar um nome reservado com parênteses é erro explícito: `tan(theta)` dá ``tan` is a known elementary function, not yet implemented as callable -- reserved names cannot be used as an indeterminate function` — mensagem real, capturada rodando o parser. Uma marca de derivada (linha ou subscrito) num nome reservado dá o mesmo erro, nunca a mensagem de "função conhecida com derivada conhecida" (essa é só para as cinco de

fato implementadas). Um nome reservado usado sem parênteses continua uma variável livre comum, sem tratamento especial nenhum — só a forma `NOME(...)` é afetada.

A derivada de uma função indeterminada de **uma variável só** usa notação de linha — `f'(r)`, `f''(r)`, uma linha por ordem de derivada, exatamente como no papel. A notação de linha é ambígua para uma função de **mais de uma variável** (derivada em relação a qual delas?), então uma função como `h(t, r)` usa um subscrito entre chaves, sempre separado por vírgula, nomeando a(s) variável(is) em relação à(s) qual(is) derivar — `h_{t}(t, r)` é $\partial h / \partial t$, `h_{r}(t, r)` é $\partial h / \partial r$, `h_{t,t}(t, r)` é $\partial^2 h / \partial t^2$, `h_{t,r}(t, r)` é a parcial mista $\partial^2 h / \partial t \partial r$. A vírgula é sempre exigida dentro das chaves (nunca a forma colada `h_{tr}` que `NOME_{...}` aceita para índice de componente de métrica, capítulo 9.2) — essa gramática não tem uma carta em escopo para decompor um nome colado de volta em coordenadas, então a vírgula é o único separador. Usar notação de linha numa função de mais de um argumento é erro explícito, apontando para a forma de subscrito. Uma função *conhecida* (`sin`, `exp`, ...) nunca aceita marca de derivada nenhuma — `sin'(x)` é erro, nomeando `diff(sin(x), x)` como a forma real de obter essa derivada.

Derivar uma expressão que contenha uma função indeterminada produz a regra da cadeia correta, generalizada sobre a ordem em vez de fixada numa função irmã (ao contrário de `sin`, cuja derivada é sempre literalmente `cos`): a derivada de `f` de ordem `k` é `f` de ordem `k+1`. `diff(f(r), r)` dá `f'(r)`; `diff(f(r)^2, r)` dá `2*f(r)*f'(r)` (regra da potência e regra da cadeia compondo normalmente); `diff(f(r)*g(r), r)` dá `f'(r)*g(r) + f(r)*g'(r)` (regra do produto, duas funções indeterminadas distintas, nunca colapsando uma na outra); a derivada de `f(r)` em relação a uma coordenada que não é `r` é zero — não como caso especial, mas porque a regra da cadeia multiplica por `d(r)/d(outra) = 0`, o mesmo mecanismo que já zera a derivada de qualquer variável em relação a outra. Verificado rodando o parser/motor de verdade: uma métrica com `f(r)` desconhecido é aceita (nunca mais rejeitada como `unknown function`), e consultar `christoffel` sobre ela produz símbolos reais contendo `f(r)` e `f'(r)`, tipografados de forma legível — por exemplo $\Gamma_{tt}^r = \frac{1}{2}f(r)f'(r)$.

9.11 `geodesic` — a equação geodésica

Esta consulta só **escreve** a equação — nunca a resolve. Não existe modo separado para geodésica nula/tipo-tempo, nem para parametrização não-afim: a equação $\ddot{x}^a + \Gamma_{bc}^a \dot{x}^b \dot{x}^c = 0$ é a mesma nos três casos — o que muda é a condição inicial, não a equação, e condição inicial não é algo que esta consulta recebe ou produz.

Sintaxe: `geodesic PARÂMETRO` ou `geodesic ALVO PARÂMETRO` — o nome do parâmetro afim (tipicamente `tau`) vem sempre por último; o alvo (nome de métrica/conexão), quando presente, vem antes dele, na mesma posição que ocupa em `christoffel / riemann / ricci / scalar / kretschmann` (“a geodésica de `g`, parametrizada por `tau`”). Ao contrário dessas cinco, o parâmetro não é opcional — `geodesic` sozinho, sem nome nenhum depois, é erro: ``geodesic` needs an affine parameter name, e.g. `geodesic tau` or `geodesic g tau``. `accel` (seção 9.12) compartilha exatamente essa mesma gramática e essa mesma regra de colisão — só o que a consulta retorna muda entre as duas.

Regra de colisão, obrigatória: o nome do parâmetro afim não pode coincidir com nenhuma coordenada da carta em uso, nem com nenhuma variável livre já usada na métrica/conexão (M , Q , ...). Coincidir tornaria a equação genuinamente ambígua — $t(r)$ seria, ao mesmo tempo, "a coordenada t como função do parâmetro r " e "a coordenada livre t multiplicada pela coordenada livre r " — então os dois casos são recusados de forma explícita, nunca resolvidos por adivinhação. Mensagens reais, capturadas rodando o parser contra `schwarzschild_ascii.od`:

```
$ oderom geodesic schwarzschild_ascii.od --param r
error: parse error: affine parameter `r` collides with chart coordinate `r`
-- geodesic needs a distinct name

$ oderom geodesic schwarzschild_ascii.od --param M
error: parse error: affine parameter `M` collides with a free variable already
used in this metric/connection -- geodesic needs a distinct name
```

O ponto central: duas encarnações do mesmo nome de coordenada. Dentro de uma mesma equação geodésica, r significa duas coisas diferentes ao mesmo tempo: dentro de um coeficiente de Christoffel, é a coordenada livre de sempre (a mesma que aparece na métrica); no termo de velocidade/aceleração, é r como função do parâmetro afim — a segunda encarnação, com derivada própria. As duas nunca colapsam numa só: são representadas por formas de árvore diferentes desde a construção (a mesma máquina de função indeterminada da seção 9.10, aqui aplicada ao próprio nome da coordenada), nunca o mesmo nó reinterpretado. O que muda entre os dois alvos de saída é só a **tipografia** dessa segunda encarnação — a mesma distinção que `theta / \theta` já tem para uma coordenada comum (seção 9.7): no alvo `Unicode`, por extenso, com o parâmetro explícito, porque um ponto sobreposto à letra não sobrevive em texto simples — `t'(tau)`, `r''(tau)`; no alvo `Latex`, notação de ponto, no estilo de livro, com o parâmetro implícito — `\dot{t}`, `\ddot{r}`. As duas descrevem a mesma equação; só a forma como a segunda encarnação é escrita muda com o alvo — a escolha nunca é do usuário via opção, segue o alvo, exatamente como o resto da saída já faz.

Exemplo real, a equação de t de Schwarzschild (capítulo 10, exemplo 5 tem as quatro por extenso), nos dois alvos:

```
--target unicode:
(2*M*r*t''(tau) - 2*M*r'(tau)*t'(tau) - r^2*t''(tau))/(2*M*r - r^2) = 0

--target latex:
\frac{(2 M r \ddot{t} - 2 M \dot{r} \dot{t} - r^2 \ddot{t})}{(2 M r - r^2)} = 0
```

Como só precisa de Γ (nunca de g^{ab} para subir/descer índice), `geodesic` funciona a partir de uma `connection` pura da mesma forma que `christoffel` / `riemann` / `ricci` já funcionam (capítulo 9.5) — ao contrário de `scalar` / `kretschmann`, que continuam exigindo uma métrica de verdade.

9.12 `accel` — a geodésica resolvida para a aceleração

Isto é isolamento algébrico, não solução de EDO: `accel` nunca integra, nunca produz uma trajetória, nunca escolhe uma condição inicial. Cada equação de `geodesic` é linear na sua própria segunda derivada, então "resolver para $\ddot{x}^a$ " é dividir pelo coeficiente de $\ddot{x}^a$ e mover o resto para o outro lado — nada além disso. É um comando separado de `geodesic` (que continua existindo, inalterado, produzindo a forma canônica igual a zero), nunca um modo dele.

Sintaxe idêntica a `geodesic: accel PARÂMETRO` ou `accel ALVO PARÂMETRO`, mesma regra de colisão (seção 9.11). A saída é $\ddot{x}^a = f^a(x, \dot{x})$, uma equação por coordenada, com a segunda derivada daquela própria coordenada isolada à esquerda — a forma que um integrador numérico consome diretamente, em vez da forma canônica igual a zero que `geodesic` produz.

Antes de dividir: cada equação de `geodesic` tem a forma $C_a \cdot \ddot{x}^a + R_a = 0$, onde C_a é o coeficiente de $\ddot{x}^a$ e R_a é o resto (nunca contém uma segunda derivada). Antes de produzir $\ddot{x}^a = -R_a/C_a$, `accel` verifica estruturalmente — nunca assume — que $\ddot{x}^a$ aparece em grau exatamente 1 na equação (nunca ao quadrado, nunca dentro de outra função, nunca em mais de um fator do mesmo produto), via `oderom_expr::isolate_linear`, uma análise puramente estrutural da árvore da expressão (nunca substituição numérica). Essa linearidade não é uma propriedade da métrica/carta escolhida — é um fato matemático sobre o que uma equação geodésica é (a própria definição $\ddot{x}^a + \Gamma^a_{bc} \dot{x}^b \dot{x}^c = 0$ já é linear, coeficiente literal 1, em $\ddot{x}^a$): nenhuma equação geodésica genuína, de nenhuma métrica ou conexão, pode deixar de satisfazê-la. Por isso, ao contrário do coeficiente (abaixo), uma violação aqui não é um `Result` que a chamadora possa capturar — é uma asserção que falha alto (`.expect()`), um pânico documentando a invariante), porque não existe entrada legítima que a alcance: só um bug na própria montagem de `geodesic_equations`, ou uma chamadora passando algo que nunca foi uma equação geodésica.

O coeficiente C_a em si é diferente: **pode**, em princípio, se anular para uma métrica/carta degenerada — isso não é uma propriedade da equação em si, é uma propriedade de qual métrica foi escolhida, então uma extensão futura do motor pode legitimamente alcançar esse caso. Por isso continua sendo um erro capturável de verdade, nomeando a coordenada, em vez de uma asserção: dada a construção atual de `geodesic_equations` (o termo de aceleração é somado uma única vez, com coeficiente literal 1) e a regra de colisão obrigatória de `geodesic / accel` (que já impede qualquer componente de Γ de conter um nó `coord''(param)`), esse coeficiente é hoje sempre exatamente 1 e nunca zero, para qualquer métrica ou conexão real — mas a verificação continua presente porque protege contra uma mudança futura em como a equação é montada, não porque o caso real precise dela hoje; se algum dia disparar, a mensagem nomeia a coordenada envolvida (`the geodesic equation for coordinate `r` has a second-derivative coefficient that is identically zero`) em vez de dividir por zero silenciosamente.

A saída passa pela mesma normalização (forma racional canônica) que o resto do motor usa — nunca uma fração dentro de fração. Exemplo real, a equação radial de

Schwarzschild (capítulo 10, exemplo 6), cujo coeficiente de Christoffel não é um monômio trivial:

```
--target unicode:
r''(tau) = (-M*r^2*r'(tau)^2 + M*r^2*t'(tau)^2 + 4*M*r^4*sin(theta)^2*phi'(tau)^2
+ 4*M*r^4*theta'(tau)^2 - 4*M^2*r*t'(tau)^2 - 4*M^2*r^3*sin(theta)^2*phi'(tau)^2
- 4*M^2*r^3*theta'(tau)^2 + 4*M^3*t'(tau)^2 - r^5*sin(theta)^2*phi'(tau)^2
- r^5*theta'(tau)^2)/(2*M*r^3 - r^4)
```

Uma única fração reduzida, não uma fração de frações, apesar do denominador `2*M*r^3 - r^4` não ser trivial — verificado tanto por inspeção quanto por um teste de aceitação dedicado (`oderom-components/tests/schwarzschild.rs`, `accel_radial_equation_of_schwarzschild_has_no_nested_fraction`).

A notação segue exatamente a mesma regra de `geodesic` (seção 9.11): ponto no alvo LaTeX dos dois lados do `=` (`\ddot{t} = ...`), forma explícita no Unicode (`t''(tau) = ...`), o parâmetro sempre implícito no LaTeX. Como só precisa de Γ , `accel` funciona a partir de uma `connection` pura exatamente como `geodesic` (capítulo 9.5).

O teste de ouro que garante que a forma resolvida e a forma canônica são a mesma equação: multiplicar `accel` de volta pelo coeficiente C_a e mover para um lado tem que reproduzir, byte a byte após normalizar, exatamente o que `geodesic` produz para a mesma métrica e o mesmo parâmetro — nunca dois cálculos independentes que só parecem concordar (`oderom-components/tests/schwarzschild.rs`, `accel_of_schwarzschild_reproduces_geodesic_when_multiplied_back`).

9.13 `einstein` — o tensor de Einstein

$G_{ab} = R_{ab} - \frac{1}{2} g_{ab} R$, totalmente covariante — aritmética literal sobre três quantidades que o programa já calcula (`ricci`, a própria métrica, e o escalar que `scalar` já produz), não uma computação de curvatura nova. Listado por componentes independentes exatamente como `ricci`: mesma tipografia de índices por nome, mesma contagem "N componentes identicamente zero", mesmo clique-para-copiar. Só a forma totalmente covariante — subir um índice para a forma mista ou contravariante é matemática nova, fora do escopo desta rodada; se um dia for pedida, é rodada à parte.

Sintaxe idêntica às primeiras cinco consultas: `einstein [ALV0]`. Ao contrário de `christoffel / riemann / ricci`, porém, **exige uma métrica de verdade** — `g_ab` aparece de forma literal na própria fórmula, e $R = g^{bd} R_{bd}$ já precisa inverter a métrica, a mesma razão que já faz `scalar / kretschmann` recusarem uma conexão pura. A partir de `connection_only.od`, mesmo tipo de erro, nome do comando trocado:

```
$ oderom einstein connection_only.od
error: `einstein` needs a metric to invert (only a connection was declared)
```

Não é a equação de campo com fonte. Esta consulta produz G_{ab} sozinho — nunca $G_{ab} = 8\pi T_{ab}$, nem nada que exija um tensor de energia-momento fornecido pelo usuário. Não há como declarar T_{ab} na linguagem hoje, e resolver a equação com fonte é um projeto à

parte. O caso de vácuo é simplesmente G_{ab} saindo identicamente zero — a própria contagem "N componentes identicamente zero" já expressa isso, sem precisar de um modo separado.

O teste de ouro: Schwarzschild é solução de vácuo ($R_{ab} = 0$, $R = 0$), então `einstein` nela tem que dar as dez componentes independentes identicamente nulas — se alguma saísse não-nula, seria um erro real, não uma peculiaridade da métrica. Verificado de fato (capítulo 10, exemplo 7).

Reissner-Nordström (elétrica, não vácuo) dá o caso oposto: componentes reais, não-nulas. Interessante à parte: o escalar de Ricci de Reissner-Nordström já é zero (seção anterior a esta, e `oderom-components/tests/reissner_nordstrom.rs`'s própria

`ricci_scalar_of_reissner_nordstrom_is_zero` — o tensor eletromagnético é de traço nulo em 4D), então a fórmula colapsa para $G_{ab} = R_{ab}$ exatamente — `einstein` e `ricci` saem *byte a byte idênticos* em Reissner-Nordström, a verificação mais direta possível de que a nova consulta reaproveita a saída real de `ricci`, não recalcula algo parecido por conta própria (`oderom-components/tests/reissner_nordstrom.rs`, `einstein_of_reissner_nordstrom_equals_ricci_since_the_scalar_vanishes`).

Um segundo caso instrutivo, à parte de Schwarzschild/Reissner-Nordström: a esfera S^2 unitária (capítulo 10, exemplo 3) tem Ricci genuinamente não-nulo ($R_{ab} = g_{ab}$, $R = 2$) — não é vácuo — e ainda assim seu tensor de Einstein também sai identicamente zero. Não é coincidência da esfera especificamente: em exatamente duas dimensões, G_{ab} é sempre identicamente nulo, um fato real sobre a própria definição do tensor de Einstein em 2D, não uma simplificação que o motor aplica. Verificado computacionalmente contra a saída real de `ricci / scalar` para S^2 (`oderom-components/tests/round_sphere_einstein.rs`), não assumido — confirma primeiro que o Ricci realmente não é nulo antes de tratar o Einstein nulo resultante como significativo.

9.14 `riccisquare / gaussbonnet` — invariantes escalares

Dois números/expressões, sem rótulo, mesma forma de `scalar / kretschmann` — nenhum dos dois é uma computação de curvatura nova, os dois são aritmética sobre o que o programa já calcula:

- `riccisquare` $\rightarrow R_{ab}R^{ab}$, o Ricci contraído consigo mesmo depois de subir os dois índices com g^{ab} .
- `gaussbonnet` $\rightarrow R_{abcd}R^{abcd} - 4R_{ab}R^{ab} + R^2$ — recombinação pura de `kretschmann`, `riccisquare` e `scalar`; em exatamente 4 dimensões é a densidade cuja integral é (proporcional a) a característica de Euler, mas a fórmula/nome fazem sentido em qualquer dimensão, e esta consulta não se restringe a 4D.

Nomes escolhidos deliberadamente sem sublinhado — `ricci_square` nunca lexaria como um token só, o mesmo problema que já descartou `geodesic_solved` em favor de `accel` (seção 9.12): `_` é o operador de subscrito deste léxico, não um caractere de identificador. `gaussbonnet` foi preferido a `euler` por ser inequívoco (não colide com característica de Euler, ângulos de Euler, etc.).

Os dois exigem uma métrica de verdade (sobem índice, precisam de g^{ab}) — recusam conexão pura com o mesmo erro padrão de `scalar / kretschmann`, nome do comando trocado.

Testes de ouro, os dois em vácuo: Schwarzschild tem Ricci nulo, então `riccisquare` nela é exatamente 0; e como os outros dois termos do Gauss-Bonnet somem em vácuo, `gaussbonnet` tem que igualar `kretschmann` byte a byte — `48*M^2/r^6`, o mesmo valor, nunca uma coincidência numérica de arredondamento (`oderom-components/tests/schwarzschild.rs`, `ricci_squared_of_schwarzschild_is_zero / gauss_bonnet_of_schwarzschild_equals_kretschmann`). Em Reissner-Nordström (não-vácuo) os dois saem genuinamente não-nulos — verificado (capítulo 10, exemplo 8).

9.15 `weyl / weylsquare` — o tensor de Weyl

A barreira de dimensão é obrigatória, leia antes de rodar num espaço 2D: `weyl / weylsquare` **recusam** em exatamente 2 dimensões — não travam, não dividem por zero, não inventam uma componente. Em 3D funcionam normalmente e dão zero (por teorema real, não por caso especial no código). Em 4D ou mais, calculam normalmente.

C_{abcd} , o tensor de Weyl (conforme), totalmente covariante, listado por componentes independentes exatamente como `riemann` — mesma tipografia de índices, mesma contagem por simetria, mesmo clique-para-copiar. Em dimensão n :

$$C_{abcd} = R_{abcd} - \frac{1}{(n-2)} (g_{ac} R_{db} - g_{ad} R_{cb} - g_{bc} R_{da} + g_{bd} R_{ca}) + \frac{R}{(n-1)(n-2)} (g_{ac} g_{db} - g_{ad} g_{cb})$$

— Riemann menos as partes de traço que o Ricci e o escalar carregam, deixando só a parte "puramente conforme". Só a forma totalmente covariante, mesma decisão de `einstein` (seção 9.13): subir índice é outra rodada. Exige uma métrica de verdade (g_{ab}/R aparecem de forma literal na fórmula) — recusa conexão pura com o erro padrão, nome do comando trocado.

A barreira de dimensão, em detalhe: os fatores $1/(n-2)$ da fórmula estouram em $n=2$ — não é o programa falhando ao calcular algo, é o próprio tensor de Weyl não sendo um objeto matemático definido em 2D (o Riemann tem uma componente independente só nessa dimensão, e não sobra nenhuma informação "puramente conforme" para separar). `weyl / weylsquare` recusam antes de dividir por zero:

```
$ oderom weyl hyperbolic_plane.od
error: the Weyl tensor is not defined in 2 dimensions (the standard
formula's 1/(n-2) term is undefined at n=2)
```

Em exatamente 3 dimensões, ao contrário, a fórmula é bem definida e o Weyl é identicamente nulo — um teorema real (em 3D o Riemann tem exatamente tantas componentes independentes quanto o Ricci, então o Ricci já determina todo o Riemann, e não sobra nada "puramente conforme"). Isso não é tratado como caso especial no código — a mesma fórmula de qualquer dimensão ≥ 3 é usada, e o resultado sai zero por conta

própria; verificado contra uma métrica 3D genuinamente curva e sem simetria máxima (não a 3-esfera, que seria conformemente plana "de graça" e testaria menos o teorema), não assumido (`oderom-components/tests/three_dimensional_weyl.rs`).

O teste de ouro: Schwarzschild é vácuo ($R_{ab}=0$, $R=0$), então os termos de correção da fórmula somem inteiramente e $C_{abcd} = R_{abcd} - \text{weyl}$ e `riemann` saem *byte a byte* *idênticos* na mesma métrica (capítulo 10, exemplo 8). Como o vácuo por si só não exercita o sinal do termo de traço ($\propto R$, que também é zero ali), a verificação independente mais forte vem de De Sitter — o único ponto fixo deste projeto com escalar de Ricci genuinamente não-nulo ($R=12H^2$, máximamente simétrico) — onde o Weyl também tem que sair zero, e sai, exercitando esse termo de verdade (`oderom-components/tests/desitter.rs` , `weyl_of_de_sitter_is_identically_zero`).

Em Reissner-Nordström (não-vácuo), Weyl é genuinamente não-nulo e diferente de Riemann — verificado tanto contra uma montagem à mão da fórmula a partir das saídas reais de `riemann / ricci / scalar` quanto contra a propriedade que de fato distingue Weyl de Riemann: ser **completamente de traço nulo** — contrair qualquer par de índices com a métrica inversa dá exatamente zero, verificado computacionalmente (não assumido), um teste matemático independente de qualquer valor que o próprio programa já produziu (`oderom-components/tests/reissner_nordstrom.rs` , `weyl_of_reissner_nordstrom_is_traceless`). Uma identidade adicional, também verificada: $R_{abcd}R^{abcd} = C_{abcd}C^{abcd} + 2 \cdot R_{ab}R^{ab}$ quando $R=0$ (o caso de Reissner-Nordström) — `kretschmann` igual a `weylsquare` mais duas vezes `riccysquare` , as três consultas concordando entre si.

9.16 Marcação de variância de índice — subir e descer índices de um tensor já calculado

Por padrão, cada consulta devolve seu tensor numa variância fixa (a tabela da seção 9.5): `riemann` sai totalmente covariante a partir de uma métrica, `ricci / einstein / weyl` também. Um marcador opcional, entre colchetes, ao final da consulta, pede a variância que o usuário quiser em vez da variância padrão — um `up` ou `down` por índice do tensor, na ordem em que os índices aparecem:

```
riemann [up,down,down,down]
ricci g [up,up]
weyl [down,down,up,down]
```

`up` sobe aquele índice com g^{ab} se ele ainda não for contravariante; `down` desce com g_{ab} se ainda não for covariante — um índice que já está na variância pedida fica como está, sem operação nenhuma. O alvo (nome da métrica/conexão) continua vindo antes do marcador, na mesma posição de sempre: `ricci g [up,up]` , não `ricci [up,up] g` . Sem o marcador, nada muda: toda consulta continua produzindo exatamente a variância que sempre produziu — o marcador é aditivo, nunca uma mudança de comportamento padrão. Transcrição real (REPL, Schwarzschild), `riemann` subindo o primeiro índice:

```

oderom> riemann [up,down,down,down]
inverting the metric...
computing Christoffel symbols...
[1] riemann [up,down,down,down] -- [OK]
R[^t,r,t,r] = 2*M/(-2*M*r^2 + r^3)
R[^t,r,r,t] = 2*M/(2*M*r^2 - r^3)
R[^t,theta,t,theta] = M/(-r)
...
232 independent components identically zero

```

O $\wedge$ na frente de t é a mesma tipografia de índice contravariante que `christoffel` já usa para Γ^a_{bc} (seção 9.4/9.7) — o mecanismo de exibição não mudou nada, só passou a receber um padrão de variância diferente do padrão fixo de sempre. Uma diferença visível e esperada: uma consulta com marcador mostra toda componente independente crua, sem o agrupamento por simetria que `riemann / ricci` sem marcador mostram (a nota "N componentes por simetria") — o grupo de simetria de um tensor de variância mista genérica não é necessariamente o mesmo do totalmente covariante, então a consulta marcada não tenta reaproveitá-lo. Os *valores* são idênticos de qualquer forma — só a contagem de linhas mostradas muda.

Quais consultas aceitam o marcador: `riemann`, `ricci`, `einstein`, `weyl` — os quatro tensores de verdade. O marcador sempre precisa de uma métrica declarada (nunca funciona a partir de uma `connection` pura, mesmo para `riemann / ricci`, que sem marcador funcionam dos dois — subir/descer índice usa g^{ab}/g_{ab} , que só existem a partir de uma métrica de verdade):

```

$ oderom-repl
oderom> ricci [up,up]
error: `ricci` needs a metric to invert (only a connection was declared)

```

O número de `up / down` tem que bater exatamente com o posto do tensor (4 para `riemann / weyl`, 2 para `ricci / einstein`) — nem menos, nem mais, nunca completado ou truncado silenciosamente:

```

oderom> riemann [up,down,down]
error: `riemann` has 4 index/indices, but the variance marker names 3 -- they must match exactly

```

Um escalar (`scalar`, `kretschmann`, `riccisquare`, `gaussbonnet`, `weylsquare`) não tem índice nenhum para subir ou descer — o marcador nesses é erro claro, não ignorado silenciosamente:

```

oderom> scalar [up]
error: `scalar` is a scalar -- it has no indices, so an index-variance marker doesn't apply. Use it on riemann, ricci, einstein, or weyl instead.

```

Christoffel não aceita marcador de variância — e essa recusa é matemática, não uma limitação do programa. Γ^a_{bc} não é um tensor: ao trocar de carta, ele se transforma com um

termo adicional que não é homogêneo (um termo com uma segunda derivada da transformação de coordenadas) — é exatamente esse termo extra que faz Γ ser uma *conexão*, e não um tensor. Um objeto que não é tensor não tem "a mesma coisa com os índices na outra posição": subir ou descer os índices de Γ com a métrica, mecanicamente, produz um conjunto de números que não se transforma nem como tensor, nem como conexão, nem como nenhum outro objeto geométrico reconhecível — um resultado com aparência de resposta legítima, mas sem significado nenhum por trás. Este projeto se recusa a produzir esse tipo de saída plausível-porém-falsa, pela mesma disciplina que já faz `scalar` recusar uma conexão pura (seção 9.5) e `weyl` recusar dimensão 2 (seção 9.15): quando uma operação é matematicamente indefinida, o motor recusa com uma mensagem que explica o porquê, em vez de calcular algo de qualquer jeito.

```
oderom> christoffel [up,down,down]
error: Christoffel is not a tensor -- it transforms with a
      non-homogeneous term, which is exactly what makes it a connection
      instead of a tensor. Raising or lowering its indices with the metric
      would not produce a tensor, a connection, or any geometric object --
      it would just be numbers with no meaning. For curvature, use
      `riemann` instead.
```

`christoffel` sem marcador nenhum continua funcionando exatamente como sempre, na sua única variância mista natural (Γ^a_{bc}) — só a *mudança* de variância é recusada, nunca a consulta em si.

9.17 `export` — tradução simbólica para Mathematica e SymPy

`QUERY := 'export' ('mathematica'|'sympy') QUERY` — a palavra-chave `export`, um dos dois alvos simbólicos, e depois **qualquer consulta** desta mesma gramática (seção 9.5), recursivamente: um escalar, um tensor por componentes, com ou sem marcador de variância (9.16), `geodesic` / `accel` com seu parâmetro. Traduz o que a consulta interna produziria para dentro deste projeto (Unicode/LaTeX) para a sintaxe de outra ferramenta simbólica, para o usuário continuar manipulando ali — nunca resimplifica nem recalcula nada: a expressão exportada é exatamente a mesma árvore já normalizada pelo motor, só a sintaxe de superfície muda.

```
export sympy kretschmann
export mathematica riemann g
export sympy geodesic g tau
export mathematica riemann [up,down,down,down]
```

Funciona nas duas superfícies — é uma construção da linguagem, não uma ação de interface: um bloco de consulta no caderno, ou a CLI, com o alvo e a consulta como dois argumentos posicionais próprios antes do arquivo:

```
oderom export mathematica kretschmann schwarzschild_ascii.od
oderom export sympy riemann schwarzschild_ascii.od --metric g
oderom export sympy geodesic schwarzschild_ascii.od --param tau
```

Um alvo desconhecido é erro claro, listando os dois nomes válidos; o mesmo vale para uma consulta desconhecida depois do alvo — nenhum dos dois é adivinhado:

```
$ oderom export octave kretschmann schwarzschild_ascii.od
error: parse error: unknown export target `octave` (expected one of:
  mathematica, sympy)
```

Sintaxe de cada alvo — verificada rodando cada exemplo abaixo dentro de um interpretador Python 3 com SymPy 1.12 de verdade (nunca só inspecionando a string; Mathematica não tem interpretador disponível neste ambiente, então sua própria verificação é só textual, contra um valor esperado conhecido — nunca confundida com prova de execução):

	Mathematica	SymPy
Potência	<code>r^6</code>	<code>r**6</code>
Multiplicação	<code>M*r</code> (ou justaposição)	<code>M*r</code> (sempre explícito — SymPy nunca aceita justaposição)
Fração literal exata	<code>1/2</code> (divisão exata entre inteiros já é o comportamento nativo de Mathematica)	<code>Rational(1, 2)</code> (nunca <code>1/2</code> nu — entre dois <code>int</code> do Python isso já vira <code>float</code> antes do SymPy sequer ver a expressão)
Funções conhecidas	<code>Sin[x]</code> , <code>Cos[x]</code> , <code>Sqrt[x]</code> , <code>Exp[x]</code> (maiúscula, colchete)	<code>sin(x)</code> , <code>cos(x)</code> , <code>sqrt(x)</code> , <code>exp(x)</code> (minúscula, parêntese)
Nome de coordenada grega	<code>theta</code> (identificador simples — nunca <code>\theta</code> , que não existe fora de LaTeX)	<code>theta</code> (idem)

Escalar (`kretschmann` de Schwarzschild), transcrição real dos dois alvos:

```
$ oderom export sympy kretschmann schwarzschild_ascii.od
M, r = symbols('M r')
48*M**2/r**6

$ oderom export mathematica kretschmann schwarzschild_ascii.od
48*M^2/r^6
```

O bloco SymPy vem com uma linha `symbols(...)` na frente, declarando toda variável livre que a expressão usa — a escolha deliberada deste projeto é que a saída seja **autossuficiente**: colada inteira numa sessão nova do Python (`from sympy import *`, e nada mais), ela roda sem passo de preparação separado. Mathematica não precisa disso — um identificador solto já é um símbolo válido lá, sem declaração prévia.

Tensor com várias componentes (`riemann`): uma atribuição por componente independente, cada uma um identificador único e válido — nunca notação de livro com índice em sobrescrito/subscrito, que não é sintaxe executável em nenhuma das duas ferramentas. SymPy separa por `_`; Mathematica por `$`, porque `_` ali é o operador de padrão `Blank`, reservado, não um separador comum:

```
$ oderom export sympy riemann schwarzschild_ascii.od
M, r, theta = symbols('M r theta')
R_t_r_t_r = 2*M/(-r**3)
# 4 components by symmetry
R_t_theta_t_theta = (M*r - 2*M**2)/r**2
# 4 components by symmetry
...
# 15 independent components identically zero
```

A anotação "N components by symmetry" e a contagem final de componentes identicamente nulas — as mesmas anotações em inglês comum da seção 9.7, nunca matemática — aparecem como comentário de verdade na sintaxe do alvo (`#` em SymPy, `(* ... *)` em Mathematica), nunca como uma linha nua no meio do bloco: uma linha nua ali quebraria a execução assim que colada (`SyntaxError` em Python) — um bug real deste próprio round, encontrado rodando a saída de `riemann` num interpretador de verdade, não só olhando a string. Um resultado com índice contravariante (marcador de variância, 9.16) recebe o mesmo tratamento, com um segmento `up` a mais por índice levantado: `R_up_t_r` (SymPy), `Ruptr` (Mathematica).

geodesic / accel: como o lado esquerdo de cada equação é uma expressão inteira, nunca um identificador solto, `=` literal seria uma atribuição inválida na linguagem de destino (`SyntaxError: cannot assign to expression here` em Python) — outro bug real deste round, também só descoberto rodando a saída de verdade. Por isso `export` usa `Eq(esquerda, direita)` em SymPy (um objeto de verdade, pronto para `solve / dsolve`) e `esquerda == direita` em Mathematica, nunca `=`:

```
$ oderom export sympy geodesic schwarzschild_ascii.od --param tau
M, r, tau, theta = symbols('M r tau theta')
Eq((2*M*r*Derivative(Function('t')(tau), tau, 2) - 2*M*Derivative(
    Function('r')(tau), tau, 1)*Derivative(Function('t')(tau), tau, 1)
    - r**2*Derivative(Function('t')(tau), tau, 2))/(2*M*r - r**2), 0)
...
```

Função indeterminada (`f(r)`, seção 9.10) — o caso em que a tradução ingênua quebraria, porque não é troca de símbolo, é mapear um conceito (uma função ainda não especificada) para a construção equivalente de cada ferramenta: SymPy usa `Function('f')(r) / Derivative(Function('f')(r), r, 1)`; Mathematica usa `f[r] / f'[r]`, sua própria notação de linha:

```
$ oderom export sympy christoffel unknown_static_spherical.od
r, theta = symbols('r theta')
Gamma_up_t_t_r = -Rational(1, 2)*Derivative(Function('f')(r), r, 1)/(-Function('f')(r))
```



```
...
$ oderom export mathematica christoffel unknown_static_spherical.od
Gamma$up$t$t$r = -1/2*f'[r]/(-f[r])
...
```

Verificação de verdade (não só sintática): substituindo $f(r) = 1 - 2M/r$ DENTRO do próprio SymPy, o Γ_{tr}^t exportado de `unknown_static_spherical.od` se reduz a $M/(r^2 - 2Mr)$ — exatamente o Γ_{tr}^t conhecido de Schwarzschild, calculado de forma totalmente independente. O nome `f` nunca é pré-vinculado a uma variável Python (`Function('f')(...)` é sempre construído embutido, a cada uso) — necessário porque o mesmo nome pode aparecer como `Symbol` comum (coeficiente de Christoffel) e como `Function(...)` de outra variável (termo de velocidade de uma geodésica) dentro da mesma equação; pré-vincular colidiria os dois.

Colisão com uma palavra reservada do alvo é detectada e documentada, nunca silenciosa. Um parâmetro ou coordenada chamado `E` (escala de energia, excentricidade — um nome plausível) colide com a constante de Euler tanto em Mathematica quanto em SymPy. `export` detecta a colisão contra a lista de nomes reservados de cada alvo (`E Pi I D N Gamma C K O` em Mathematica; palavras-chave do Python mais `I E pi oo zoo nan S N` em SymPy), renomeia (`E → E_` em SymPy, `E → E$` em Mathematica) e documenta a troca como comentário na própria saída — nunca emite algo que colidiria com a constante real, e nunca deixa o usuário descobrir sozinho por que o resultado está errado:

```
$ oderom export sympy scalar colisao.od
E_, r = symbols('E_ r')
2*E_/r**3
# renamed `E` to `E_` -- collides with a reserved SymPy name
```

No caderno/aplicativo, cada componente de um resultado `export` continua clicável individualmente (9.7) — o texto copiado é o código-fonte pronto daquela linha, nunca com o comentário misturado; a anotação (nota de simetria, de renomeação) aparece como legenda separada ao lado, exatamente como já acontece para qualquer outro resultado.

O que `export` não faz: não reimplementa nem re-simplifica nada — a árvore exportada é a mesma que o motor já normalizou, só muda de sintaxe; não produz LaTeX (já existe, `--target latex`); e, nesta rodada, só cobre os dois alvos simbólicos. Alvos numéricos (C++, Fortran, Julia) são rodadas futuras, de natureza diferente (gerar código para compilar/rodar, não para continuar manipulando simbolicamente) — não implementados aqui.

10. Exemplos completos

Os oito exemplos abaixo foram executados de verdade pela linha de comando (capítulo 11) — o mesmo motor de cálculo que o caderno usa (`oderom-cli::commands / oderom-components::curvature`) — e a saída abaixo foi copiada diretamente do terminal, não reescrita de memória. Os três primeiros arquivos são as próprias fixtures deste repositório

(`oderom-cli/tests/fixtures/`); o quarto é uma métrica nova, escrita para esta seção e também executada; o quinto (`geodesic`), o sexto (`accel`), o sétimo (`einstein`) e o oitavo (`weyl / riccisquare`) reusam a métrica do primeiro (e, para `einstein / weyl` , também a do segundo). Um candidato do pedido original — um exemplo de holonomia — não entrou: como o capítulo 9.5 registra, não há consulta de holonomia na linguagem hoje, então não haveria um `.od` real para mostrar.

Exemplo 1 — Schwarzschild (`schwarzschild_ascii.od`)

```
manifold M dim 4
bundle TM on M dim 4
chart schw on M coords (t, r, theta, phi)
metric g on schw bundle TM {
  [t,t] = -(1 - 2*M/r),
  [r,r] = 1/(1 - 2*M/r),
  [theta,theta] = r^2,
  [phi,phi] = r^2 * sin(theta)^2
}
```

`ricci`:

10 independent components identically zero

`kretschmann`:

$48M^2/r^6$

Demonstra a forma básica de uma declaração completa (variedade, fibrado, carta, métrica) e as cinco consultas na sua forma mais simples, com um só parâmetro (`M`). O Ricci nulo (solução de vácuo) ao lado de um Kretschmann não-nulo é o próprio ponto do exemplo: a variedade é curva mesmo onde o tensor de Ricci desaparece inteiro.

Exemplo 2 — Reissner-Nordström (`reissner_nordstrom.od`)

```
manifold M dim 4
bundle TM on M dim 4
chart schw on M coords (t, r, theta, phi)
metric g on schw bundle TM {
  [t,t] = -(1 - 2*M/r + Q^2/r^2),
  [r,r] = 1/(1 - 2*M/r + Q^2/r^2),
  [theta,theta] = r^2,
  [phi,phi] = r^2 * sin(theta)^2
}
```

`kretschmann` (unicode, padrão):

$(-96MQ^2r + 48M^2r^2 + 56Q^4)/r^8$

a mesma consulta com `--target latex` :

```
\frac{-96 M Q^2}{r} + 48 M^2 r^2 + 56 Q^4}{r^8}
```

Demonstra uma métrica de dois parâmetros (M e Q) — três termos em $f(r)$ em vez de dois — e o alvo LaTeX produzindo `\frac{...}{...}` /expoentes reais a partir da mesma árvore. O resultado bate com a forma fechada do livro-texto, $48M^2/r^6 - 96MQ^2/r^7 + 56Q^4/r^8$, e com o teste de aceitação permanente deste repositório (`oderom-components/tests/reissner_nordstrom.rs`).

Exemplo 3 — esfera redonda S^2 (dimensão 2, curvatura não-nula)

```
manifold S2 dim 2
bundle TS2 on S2 dim 2
chart standard on S2 coords (theta, phi)
metric g on standard bundle TS2 {
  [theta,theta] = 1,
  [phi,phi] = sin(theta)^2
}
```

`ricci` :

```
Ricci[theta,theta] = 1
Ricci[phi,phi] = sin(theta)^2
1 independent component identically zero
```

`scalar` :

2

`kretschmann` :

4

Demonstra que a dimensão da variedade não é fixa em 4 — aqui é 2 — e que a dependência angular (`sin(theta)` num componente da métrica) pode ser a *única* fonte de curvatura, sem massa nem parâmetro nenhum: ao contrário dos dois exemplos anteriores, o Ricci aqui não é zero. Escalar de Ricci 2 (curvatura gaussiana 1) é o valor de livro-texto para a esfera unitária.

Exemplo 4 — conexão sem métrica (`connection_only.od`)

```
manifold M dim 2
bundle TM on M dim 2
chart flat2 on M coords (x, y)
connection Gamma on flat2 {
```

```
[x,y,y] = x
}
```

`riemann` (funciona, forma mista, sem baixar índice):

```
R[^x,y,x,y] = 1
R[^x,y,y,x] = -1
14 independent components identically zero
```

`kretschmann` (recusa):

```
error: `kretschmann` needs a metric to invert (only a connection was declared)
```

Demonstra a porta de entrada alternativa `connection`, sem `metric` nem `bundle` algum, e o limite real dela: `christoffel` / `riemann` / `ricci` funcionam direto a partir de Γ , mas `kretschmann` / `scalar` — que precisam inverter uma métrica para subir índice — recusam com um erro específico em vez de travar ou adivinhar. É também o único exemplo em que `riemann` mostra a forma mista R^a_{bcd} (componentes não agrupadas por simetria de tensor) em vez da forma totalmente covariante dos exemplos 1-3.

Exemplo 5 — a equação geodésica de Schwarzschild (`geodesic tau`)

Mesma métrica do exemplo 1 (`schwarzschild_ascii.od`); a consulta é `geodesic tau`, alvo `Unicode` (padrão) — quatro equações, uma por coordenada, na mesma ordem da carta (`t`, `r`, `theta`, `phi`):

```
(2*M*r*t''(tau) - 2*M*r'(tau)*t'(tau) - r^2*t''(tau))/(2*M*r - r^2) = 0
(M*r^2*r'(tau)^2 - M*r^2*t'(tau)^2 + 2*M*r^3*r''(tau) - 4*M*r^4*sin(theta)^2*phi'(tau)^2
- 4*M*r^4*theta'(tau)^2 + 4*M^2*r*t'(tau)^2 + 4*M^2*r^3*sin(theta)^2*phi'(tau)^2
+ 4*M^2*r^3*theta'(tau)^2 - 4*M^3*t'(tau)^2 - r^4*r''(tau) + r^5*sin(theta)^2*phi'(tau)^2
+ r^5*theta'(tau)^2)/(2*M*r^3 - r^4) = 0
(-r*sin(theta)*cos(theta)*phi'(tau)^2 + r*theta''(tau) + 2*r'(tau)*theta'(tau))/r = 0
(r*sin(theta)*phi''(tau) + 2*r*cos(theta)*phi'(tau)*theta'(tau) + 2*sin(theta)*phi'(tau)*r'(tau))
/(r*sin(theta)) = 0
```

A mesma consulta com `--target latex` — mesmas quatro equações, só a segunda encarnação de cada coordenada (seção 9.11) muda de notação, de explícita para ponto:

```
\frac{(2 M r \ddot{t} - 2 M \dot{r} \dot{t} - r^2 \ddot{t})}{(2 M r - r^2)} = 0
\frac{(-r \sin(\theta) \cos(\theta) \dot{\phi}^2 + r \ddot{\theta} + 2 \dot{r} \dot{\theta})}{r \sin(\theta)} = 0
```

A primeira equação (a de `t`) pode ser conferida à mão a partir da própria saída de `christoffel` sobre a mesma métrica (exemplo 1): os únicos Γ^t_{bc} não-nulos são Γ^t_{tr} e Γ^t_{rt} (iguais, por simetria dos dois índices baixos), ambos $-M/(2M*r - r^2)$. Substituindo na forma fechada $\ddot{x}^t + \Gamma^t_{bc} \dot{x}^b \dot{x}^c = 0$ com só esses dois termos ($\ddot{t} + 2 \cdot \Gamma^t_{tr} \cdot \dot{t} \cdot \dot{r} = 0$) e pondo sobre o mesmo denominador dá exatamente a primeira linha acima — a mesma verificação que o teste de aceitação deste round faz por código (`oderom-components/tests/`

`schwarzschild.rs`,
`geodesic_of_schwarzschild_matches_a_hand_assembly_from_this_runs_own_christoffel`),
 nunca contra um valor de livro-texto decorado.

Demonstra a distinção central da seção 9.11 dentro de uma equação real: o denominador de cada equação (o coeficiente de Christoffel) permanece escrito na coordenada livre de sempre (`r`, `theta`), a mesma forma que a métrica do exemplo 1 usa; os termos de velocidade/aceleração (`t'(tau)` / `\dot t`, etc.) são a segunda encarnação, função do parâmetro afim — as duas convivendo na mesma equação sem nunca se confundir. Nenhuma das quatro equações é resolvida aqui — a consulta escreve, não integra.

Exemplo 6 — a mesma geodésica resolvida para a aceleração (`accel tau`)

Mesma métrica do exemplo 1; a consulta é `accel tau` em vez de `geodesic tau` — as mesmas quatro equações do exemplo 5, agora com a segunda derivada de cada coordenada isolada à esquerda:

```
t''(tau) = 2*M*r'(tau)*t'(tau)/(2*M*r - r^2)
r''(tau) = (-M*r^2*r'(tau)^2 + M*r^2*t'(tau)^2 + 4*M*r^4*sin(theta)^2*phi'(tau)^2
+ 4*M*r^4*theta'(tau)^2 - 4*M^2*r*t'(tau)^2 - 4*M^2*r^3*sin(theta)^2*phi'(tau)^2
- 4*M^2*r^3*theta'(tau)^2 + 4*M^3*t'(tau)^2 - r^5*sin(theta)^2*phi'(tau)^2
- r^5*theta'(tau)^2)/(2*M*r^3 - r^4)
theta''(tau) = (r*sin(theta)*cos(theta)*phi'(tau)^2 - 2*r'(tau)*theta'(tau))/r
phi''(tau) = (-2*r*cos(theta)*phi'(tau)*theta'(tau) - 2*sin(theta)*phi'(tau)*r'(tau))
/(r*sin(theta))
```

A mesma consulta com `--target latex` — ponto nos dois lados do `=`, parâmetro implícito:

```
\ddot{t} = \frac{2 M \dot{r} \dot{t}}{(2 M r - r^2)}
\ddot{\theta} = \frac{(r \sin(\theta) \cos(\theta) \dot{\phi}^2 - 2 \dot{r} \dot{\theta})}{r \sin(\theta)}
```

Conferindo a equação de `t` à mão: multiplicando `t''(tau) = 2*M*r'(tau)*t'(tau)/(2*M*r - r^2)` de volta pelo denominador (`2*M*r - r^2`) e movendo tudo para o mesmo lado dá `2*M*r*t'' - r^2*t'' - 2*M*r'*t' = 0`, que é exatamente o exemplo 5 reorganizado (`2*M*r*t''(tau) - 2*M*r'(tau)*t'(tau) - r^2*t''(tau) = 0`, sobre o mesmo denominador) — a mesma verificação, por código, que `oderom-components/tests/schwarzschild.rs`'s `accel_of_schwarzschild_reproduces_geodesic_when_multiplied_back` faz para as quatro coordenadas, nunca só a de `t`. Note que o coeficiente de `t'' / r'' / ...` é sempre exatamente 1 aqui (seção 9.12 explica por quê) — a equação radial parece mais complicada só porque o coeficiente de Christoffel que ela isola contra, `2*M*r^3 - r^4`, não é um monômio simples, não porque o coeficiente da própria segunda derivada mude.

Exemplo 7 — o tensor de Einstein: vácuo contra eletrovácuo (`einstein`)

Mesma métrica do exemplo 1, Schwarzschild — solução de vácuo:

10 independent components identically zero

Mesma métrica do exemplo 2, Reissner-Nordström — não é vácuo (o campo elétrico é a fonte), e `einstein` mostra isso diretamente, quatro componentes reais e não-nulas:

```
G[t,t] = (2*M*Q^2*r - Q^2*r^2 - Q^4)/(-r^6)
G[r,r] = Q^2/(2*M*r^3 - Q^2*r^2 - r^4)
G[theta,theta] = Q^2/r^2
G[phi,phi] = Q^2*sin(theta)^2/r^2
6 independent components identically zero
```

Comparando com `ricci` e `scalar` na mesma métrica (capítulo 9.5): `scalar` dá `0`, e as quatro componentes de `ricci` acima são **byte a byte idênticas** às de `einstein` — exatamente o que a seção 9.13 antecipa ($G_{ab} = R_{ab} - \frac{1}{2}g_{ab} \cdot 0 = R_{ab}$). Não é coincidência de arredondamento nem de forma de exibição: é a fórmula colapsando de verdade porque o escalar desta métrica é zero.

Demonstra ao mesmo tempo o teste de ouro (vácuo dá zero) e o caso não-trivial (eletrovácuo não dá), e a verificação mais direta de todas — comparar contra as próprias saídas de `ricci / scalar` do programa para a mesma métrica, nunca contra um valor de livro-texto copiado à mão.

Exemplo 8 — o tensor de Weyl e um invariante escalar (`weyl` , `riccisquare`)

Mesma métrica do exemplo 1, Schwarzschild — vácuo, então Weyl tem que igualar Riemann byte a byte:

```
C[t,r,t,r] = 2*M/(-r^3)
(4 components by symmetry)
C[t,theta,t,theta] = (M*r - 2*M^2)/r^2
(4 components by symmetry)
C[t,phi,t,phi] = (M*r*sin(theta)^2 - 2*M^2*sin(theta)^2)/r^2
(4 components by symmetry)
C[r,theta,r,theta] = M/(2*M - r)
(4 components by symmetry)
C[r,phi,r,phi] = M*sin(theta)^2/(2*M - r)
(4 components by symmetry)
C[theta,phi,theta,phi] = 2*M*r*sin(theta)^2
(4 components by symmetry)
15 independent components identically zero
```

Comparando com `riemann` na mesma métrica (capítulo 9.5): as seis componentes acima, os mesmos coeficientes, a mesma contagem de zero — **idêntico**, nunca só parecido.

Mesma métrica do exemplo 2, Reissner-Nordström — não é vácuo, então Weyl difere de Riemann de verdade:

```
C[t,r,t,r] = (-2*M*r + 2*Q^2)/r^4
(4 components by symmetry)
C[r,theta,r,theta] = (-M*r + Q^2)/(-2*M*r + Q^2 + r^2)
(4 components by symmetry)
15 independent components identically zero
```

`riccisquare` na mesma métrica:

```
4*Q^4/r^8
```

A barreira de dimensão, no arquivo 2D deste próprio repositório (`hyperbolic_plane.od` , capítulo 9.2):

```
$ oderom weyl hyperbolic_plane.od
error: the Weyl tensor is not defined in 2 dimensions (the standard
formula's 1/(n-2) term is undefined at n=2)
```

Demonstra a seção 9.15 inteira num só exemplo: o teste de ouro em vácuo (Weyl = Riemann), o caso genuinamente não-trivial (Reissner-Nordström, Weyl $\neq$ Riemann, invariante não-nulo), e a recusa limpa por dimensão — nunca um valor calculado às cegas onde a matemática não permite.

11. Uso pela linha de comando

Existem **dois** binários de linha de comando, separados entre si e do aplicativo de caderno: `oderom` (execução única, um comando por chamada de processo) e `oderom-repl` (sessão interativa, de longa duração). Ambos usam a mesma gramática do capítulo 9 e o mesmo motor de cálculo do caderno — nenhuma construção da linguagem muda.

11.1 `oderom` — execução única

A mensagem de uso real (`oderom-cli/src/error.rs` , `CliError::Usage`):

```
usage: oderom canon [--prelude PATH] "<expression>"
or: oderom {christoffel|riemann|ricci|scalar|kretschmann|einstein|
riccisquare|gaussbonnet|weyl|weylsquare} FILE
[--metric NAME | --connection NAME]
[--target unicode|latex|json|mathematica|sympy]
[--max-lines N] [--max-nodes N] [--max-denominator-degree N]
[--timeout SECONDS]
or: oderom {geodesic|accel} FILE --param NAME
[--metric NAME | --connection NAME]
[--target unicode|latex|json|mathematica|sympy]
[--max-lines N] [--max-nodes N] [--max-denominator-degree N]
[--timeout SECONDS]
or: oderom export {mathematica|sympy} {christoffel|riemann|ricci|
scalar|kretschmann|geodesic|accel|einstein|riccisquare|
gaussbonnet|weyl|weylsquare} FILE
[--metric NAME | --connection NAME] [--param NAME]
```

```
[--max-nodes N] [--max-denominator-degree N] [--timeout SECONDS]
or: oderom load NAME
```

Uso típico, com uma das dez consultas de componentes/escalar do capítulo 9.5 e um arquivo `.od`:

```
oderom kretschmann minha_metrica.od
oderom riemann minha_metrica.od --metric g2 --target latex
oderom einstein minha_metrica.od
oderom weyl minha_metrica.od
oderom riccisquare minha_metrica.od
```

`geodesic` / `accel` (seções 9.11/9.12) são as únicas duas das doze com uma flag própria — `--param`, obrigatória, o nome do parâmetro afim (o `PARÂMETRO` da gramática de dentro do caderno, aqui como flag em vez de segundo identificador posicional):

```
oderom geodesic schwarzschild_ascii.od --param tau
oderom accel schwarzschild_ascii.od --param tau
oderom accel schwarzschild_ascii.od --param tau --target latex
```

`--timeout` tem padrão de 30s (`DEFAULT_TIMEOUT`, `commands.rs`). Cada estágio do cálculo (inverter a métrica, Christoffel, Riemann, ..., montar a equação geodésica) é anunciado em `stderr` conforme começa — se o tempo esgotar, a última linha anunciada é exatamente o estágio em andamento. `--max-nodes` (padrão 20000) e `--max-denominator-degree` (padrão 30) são guardas independentes de tamanho de árvore, verificadas a cada estágio.

`oderom canon [--prelude PATH] "expressão"` é um terceiro modo, para o nível abstrato do Marco 1 (contração de índices, `head / symmetry` do capítulo 9.3) — separado das doze consultas de curvatura, opera sobre `prelude.od` por padrão.

`oderom load NAME` é um quarto modo, sem `FILE`: imprime as declarações de uma métrica conhecida da galeria (capítulo 13) diretamente em `stdout`, como um arquivo `.od` completo e válido — não uma construção da gramática do capítulo 9, só mais um jeito de começar um arquivo sem digitá-lo à mão. Compõe normalmente com qualquer outra consulta via redirecionamento:

```
oderom load antidesitter > antidesitter.od
oderom scalar antidesitter.od
-12*H^2
```

Um nome desconhecido falha e lista, na própria mensagem de erro, todos os nomes conhecidos:

```
$ oderom load kerr
error: unknown gallery entry `kerr` (known: desitter, antidesitter, frw,
      schwarzschild, reissnernordstrom)
```


11.2 `oderom-repl` — sessão interativa

Comandos reais, exatamente como o REPL os lista (`oderom-repl/src/main.rs`): `:load` `:reload` `:defs` `:entries` `:recompute` `:save` `:timeout` `:quit`. Uma linha sem `:` na frente é uma consulta (capítulo 9.5), executada imediatamente contra o arquivo carregado. Transcrição real de uma sessão:

```
$ oderom-repl
ODEROM REPL. :load <file.od> to start, :quit or Ctrl+D to leave.
oderom> :load schwarzschild_ascii.od
ok: 2 definition(s), 9ms
oderom> christoffel
inverting the metric...
computing Christoffel symbols...
[1] christoffel -- [OK] (147ms)
Gamma[^t,t,r] = -M/(2*M*r - r^2)
...
oderom> :entries
[1] christoffel -- [OK] (147ms)
```

`:reload` relê o mesmo arquivo do disco (útil depois de editá-lo num editor externo) e marca como obsoletas as entradas que dependiam do que mudou — mesmo princípio de obsolescência do capítulo 5. `:recompute <n>` ou `:recompute stale` recalcula uma entrada específica ou todas as obsoletas.

11.3 Cancelamento e limite de tempo: os dois binários se comportam diferente

Esta é a diferença real, verificada rodando os dois contra a mesma métrica que nunca termina sozinha (`g_tt / g_rr` não recíprocos — o mesmo caso que motivou o cancelamento no caderno, capítulo 6):

oderom (execução única): `--timeout` encerra o *processo inteiro* quando esgota, com uma mensagem nomeando o estágio em andamento:

```
$ oderom kretschmann metrica_nao_reciproca.od --timeout 3
inverting the metric...
computing Christoffel symbols...
computing the Riemann tensor (mixed)...
error: timed out after 3s -- last stage in progress:
  `computing the Riemann tensor (mixed)`
```

Não há como continuar depois disso — não há "depois", já que cada chamada do binário é um cálculo só. O processo termina; se o cálculo estivesse rodando numa thread em segundo plano, ela morre junto com o processo, não por ter sido avisada para parar.

oderom-repl (sessão longa): `Ctrl+C` (ou `:timeout` esgotando, mesmo padrão de 30s) cancela só a consulta em andamento — o processo continua de pé, pronto para a próxima consulta, exatamente como o caderno nunca perde a janela por causa de um bloco cancelado (capítulo 6):

```
oderom> :timeout 3
timeout set to 3s
oderom> kretschmann
inverting the metric...
computing Christoffel symbols...
timed out after 3s -- cancelling...
[1] kretschmann -- [CANCELLED]
oderom> scalar
inverting the metric...
computing Christoffel symbols...
timed out after 3s -- cancelling...
[2] scalar -- [CANCELLED]
```

A segunda consulta, depois da primeira ser cancelada, roda normalmente — a sessão sobrevive. É o mesmo mecanismo de cancelamento profundo do caderno (`oderom_expr::run_cancellable`, verifica dentro do próprio laço de normalização, não só entre componentes) — o REPL e o caderno passam pela mesma função `oderom-session::run_query`; o binário `oderom` autônomo não passa por ela, só por um limite de parede sem cancelamento profundo, por isso a diferença.

12. Referência rápida

Tecclas de execução

Tecla	Ação
<code>Shift</code> + <code>Enter</code>	Executa; foco vai para o próximo bloco (cria um se for o último).
<code>Ctrl</code> + <code>Enter</code>	Executa; foco permanece no bloco.
<code>Alt</code> + <code>Enter</code>	Executa; insere um bloco novo abaixo e foca nele.

Estados de bloco

Indicador	Estado
<code>[]</code>	Nunca executado
<code>[n]</code>	Executado, atual (n = ordem de execução)
<code>[n]</code> âmbar + tarja + <code>u</code>	Executado, obsoleto
—	Em execução (cancelável)
—	Cancelado

Construções da linguagem

Construção	Sintaxe
------------	---------

<code>manifold</code>	<code>manifold NOME dim N</code>
<code>bundle</code>	<code>bundle NOME on VARIEDADE dim N</code>
<code>head</code> (abstrato, Marco 1)	<code>head NOME : FIBRADO[*], ... [symmetry antisymmetric symmetric CICLOS+SINAL]</code>
<code>chart</code>	<code>chart NOME on VARIEDADE coords (c1, c2, ...)</code>
<code>metric</code>	<code>metric NOME on CARTA bundle FIBRADO { [i,j] = EXPR, ... }</code>
<code>connection</code>	<code>connection NOME on CARTA { [i,j,k] = EXPR, ... }</code>
alias (9.9)	<code>NOME := EXPR</code> — açúcar sintático, expandido antes do motor; uso é <code>NOME</code> sem parênteses, nunca <code>NOME(...)</code> (sempre chamada de função)
função indeterminada (9.10)	<code>f(r)</code> , <code>h(t, r)</code> — símbolo opaco, nunca avaliado; representa e deriva, nunca resolve
derivada (9.10)	<code>f'(r)</code> , <code>f''(r)</code> (uma variável); <code>h_{t}(t,r)</code> , <code>h_{t,r}(t,r)</code> (subscrito, várias variáveis)
nomes reservados (9.10)	<code>tan cot sec csc tanh coth sech csch asin arcsin acos arccos atan arctan asinh arcsinh acosh arccosh atanh arctanh log ln sqrt atan2 abs</code> — rejeitados com parênteses, nunca aceitos como função indeterminada
<code>christoffel</code>	<code>christoffel [ALV0] → Γ^a_{bc}</code>
<code>riemann</code>	<code>riemann [ALV0] → R_{abcd} (métrica) ou R^a_{bcd} (conexão pura)</code>
<code>ricci</code>	<code>ricci [ALV0] → R_{bd}</code>
<code>scalar</code>	<code>scalar [ALV0] → R</code> (precisa de métrica)
<code>kretschmann</code>	<code>kretschmann [ALV0] → $R_{abcd}R^{abcd}$</code> (precisa de métrica)
<code>geodesic</code> (9.11)	<code>geodesic PARÂMETRO</code> ou <code>geodesic ALVO PARÂMETRO → $\ddot{x}^a + \Gamma^a_{bc}\dot{x}^b\dot{x}^c = 0$</code> , uma por coordenada (funciona a partir de <code>connection</code>); parâmetro não pode coincidir com coordenada nem variável livre da métrica/conexão
<code>accel</code> (9.12)	<code>accel PARÂMETRO</code> ou <code>accel ALVO PARÂMETRO → $\ddot{x}^a = f^a(x, \dot{x})$</code> , a mesma equação de <code>geodesic</code> resolvida para a aceleração (funciona a partir de <code>connection</code>); mesma regra de colisão de <code>geodesic</code>
<code>einstein</code> (9.13)	<code>einstein [ALV0] → $G_{ab} = R_{ab} - \frac{1}{2}g_{ab}R$</code> (precisa de métrica)
<code>riccisquare</code> (9.14)	<code>riccisquare [ALV0] → $R_{ab}R^{ab}$</code> (precisa de métrica)
<code>gaussbonnet</code> (9.14)	<code>gaussbonnet [ALV0] → $R_{abcd}R^{abcd} - 4R_{ab}R^{ab} + R^2$</code> (precisa de métrica)

<code>weyl</code> (9.15)	<code>weyl [ALV0]</code> → C_{abcd} , o tensor de Weyl (precisa de métrica; recusa em dimensão 2)
<code>weylsquare</code> (9.15)	<code>weylsquare [ALV0]</code> → $C_{abcd}C^{abcd}$ (precisa de métrica; recusa em dimensão 2)
marcador de variância (9.16)	sufixo opcional em <code>riemann / ricci / einstein / weyl</code> : <code>riemann [up,down,down,down]</code> , <code>ricci ALV0 [up,up]</code> — um <code>up / down</code> por índice do tensor, sempre depois do alvo; exige métrica; recusa em <code>christoffel</code> (não é tensor) e nos cinco escalares (sem índice)
<code>export</code> (9.17)	<code>export mathematica QUERY</code> ou <code>export sympy QUERY</code> , envolvendo qualquer uma das doze consultas acima (com marcador de variância ou não) → a mesma saída traduzida para a sintaxe de Mathematica/SymPy; funciona no caderno e na CLI (<code>oderom export {mathematica sympy} COMANDO FILE</code>); alvo desconhecido ou consulta desconhecida são erro claro; colisão com palavra reservada do alvo (<code>E</code> , <code>Pi</code> , <code>lambda</code> , ...) é renomeada e documentada como comentário
<code>load</code> (11.1, 13)	não é construção da gramática — botão Galeria no caderno (cola blocos) ou <code>oderom load NAME</code> na CLI (imprime em stdout); nomes: <code>desitter</code> , <code>antidesitter</code> , <code>frw</code> , <code>schwarzschild</code> , <code>reissnernordstrom</code>
operadores	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>^n</code> (n inteiro), justaposição = produto
funções	<code>sin(x)</code> , <code>cos(x)</code> , <code>exp(x)</code> , <code>sinh(x)</code> , <code>cosh(x)</code> — únicas aceitas hoje

13. A galeria de espaços-tempo

A galeria é uma lista de métricas conhecidas que podem ser carregadas no caderno sem digitá-las à mão. O botão **Galeria**, no alto da janela (capítulo 3), abre um painel com cada entrada disponível, seu nome, uma linha descrevendo o espaço-tempo e o invariante de curvatura que o caracteriza. Clicar em uma entrada carrega essa métrica.

Carregar cola texto, não executa nada. `load` insere as declarações da métrica escolhida como blocos novos, editáveis, ao final do caderno — exatamente o texto que um exemplo digitado à mão teria, nunca uma métrica opaca por trás de um botão. Nada é executado automaticamente (capítulo 3, "nada recalcula sozinho"); os blocos aparecem como `[ ]`, prontos para `Shift` + `Enter`. Blocos já existentes no caderno nunca são tocados.

Toda métrica traz parâmetros simbólicos (`M`, `H`, `a`, `Q`, ...). Fixar um valor não precisa de nenhum mecanismo novo: edite o texto colado diretamente, ou declare um alias em um bloco próprio (capítulo 9.9) — por exemplo, `M := 1` antes de usar a métrica.

Cada entrada da galeria é também um teste de aceitação do motor: sua curvatura foi computada e conferida contra um resultado fechado conhecido antes de entrar na lista (ver a suíte de testes de `oderom-notebook` e `oderom-components`). Uma galeria que carregasse métricas sem computar corretamente seria pior que nenhuma galeria — por isso a lista abaixo é deliberadamente pequena: só entram métricas verificadas.

Nome	O que é	Invariante verificado
<code>desitter</code>	De Sitter, fatiamento plano (curvatura constante positiva)	$R = 12H^2$ (constante); tensor de Weyl identicamente nulo
<code>antidesitter</code>	Anti-de Sitter, coordenadas de Poincaré (curvatura constante negativa)	$R = -12H^2$ (constante); $R_{ab} = -3H^2 g_{ab}$
<code>frw</code>	Friedmann-Robertson-Walker espacialmente plano, fator de escala $a(t)$ genérico	$R = 6(a''(t)/a(t) + a'(t)^2/a(t)^2)$
<code>schwarzschild</code>	Vácuo, estático, simétrico esférico	$R_{ab} = 0$; Kretschmann $= 48M^2/r^6$
<code>reissnernordstrom</code>	Simétrico esférico, carregado (eletrovácuo — não é vácuo puro)	$R = 0$, mas $R_{ab} \neq 0$ (traço nulo, tensor de Ricci não)

A entrada `frw` merece nota à parte: $a(t)$ é uma função indeterminada (capítulo 9.10) usada diretamente dentro de um componente da métrica, não apenas dentro de uma equação de geodésica como em exemplos anteriores. O motor deriva Γ , R_{ab} e R corretamente em termos de $a(t)$, $a'(t)$ e $a''(t)$ sem que o usuário precise dizer o que $a(t)$ é.

A restrição a métricas diagonais não existe mais no motor (ver [DESIGN-M2.md](#), seção "D-M2.1 revisitada"). A inversão da métrica hoje detecta a estrutura de g em três níveis — diagonal (caminho rápido inalterado), diagonal por blocos (cada bloco desacoplado invertido isoladamente, ex.: o bloco 2×2 $g_{t\phi}$ de Kerr) e geral (adjunta/cofatores) — e verifica $g_{ab}g^{bc} = \delta^a_c$ antes de aceitar o resultado. `ComponentError::NonDiagonalMetric` continua existindo como o erro que a função interna de caminho rápido (`metric_inverse_diagonal`) devolve se chamada diretamente numa métrica não-diagonal, mas não é mais o que o motor faz para uma consulta real.

Ainda assim, **Kerr e Gödel continuam fora da galeria — e não porque a inversão falhe.** Para os dois, a inversão da métrica em si é rápida e correta (microsegundos a poucas dezenas de milissegundos; nenhuma métrica de nenhum dos dois recusa ou trava nesse passo). O que bloqueia cada um é a simplificação da curvatura resultante esbarrando num limite do normalizador de `oderom-expr`, não a inversão: o Christoffel/Riemann do Kerr não terminam em tempo prático contra o denominador genuinamente bivariado da métrica; o escalar de Ricci de Gödel calcula para um valor algebricamente correto mas que o normalizador não reduz à forma fechada. Nenhuma entrada de galeria entra sem um teste de curvatura verificado de verdade — por isso os dois ficam de fora, documentados ([DESIGN-RATIONAL-FORM.md](#), seção 7, tem o detalhe completo de cada limite), sem previsão de quando isso muda.

Pela linha de comando (capítulo 11.1), `oderom load NAME` dá acesso ao mesmo catálogo: imprime as declarações da métrica em `stdout`, como um arquivo `.od` completo, em vez de colar blocos — a diferença faz sentido, já que a CLI não tem lista de blocos para colar, mas o catálogo é exatamente um só (`oderom-cli/src/gallery.rs`), compartilhado pelas

duas superfícies. Redirecionado para um arquivo, o resultado funciona com qualquer outra consulta sem edição:

```
oderom load schwarzschild > schwarzschild.od
oderom kretschmann schwarzschild.od
```

O REPL (capítulo 11.2) não ganhou um comando próprio nesta rodada: seu `:load <arquivo.od>` carrega um arquivo do disco para a sessão, não um nome da galeria — quem usa o REPL e quer uma métrica da galeria hoje precisa passar pelo binário `oderom` primeiro (`oderom load NAME > arquivo.od`, depois `:load arquivo.od` no REPL). Uma extensão de `:load` para aceitar também um nome da galeria é trabalho razoável para uma rodada futura, não incluído aqui.

ODEROM · manual do usuário. Todos os capítulos verificados contra o código-fonte e exemplos executados nesta revisão.